

Víncius Santos Anjos da Silva

**Modelagem baseada em agentes paralelizada em CUDA:
O impacto da COVID-19 em áreas urbanas no Brasil**

Universidade Federal Fluminense - UFF

Campus Volta Redonda

Instituto de Ciências Exatas

Orientador: Aquino Lauri de Espíndola

25 de novembro de 2022

Resumo

Para simular uma epidemia em uma população ao longo de um tempo utiliza-se de modelos epidemiológicos, dentre eles está o modelo baseado em agentes (ABM), no qual cada agente é equivalente a um indivíduo na população e possui diversas características, entretanto modelar áreas que possuam grandes populações e diversos parâmetros demanda grande capacidade computacional. Com o intuito de reduzir o tempo das simulações, esse trabalho tem como objetivo implementar uma modelagem baseada em agentes paralelizada utilizando CUDA, uma API que estende as linguagens C/C++ para utilizar os recursos computacionais de GPUs e criar programas paralelizáveis.

Palavras-chaves: Covid-19, modelagem baseada em agentes, programação paralela, GPU, CUDA

Sumário

Sumário	3
1 INTRODUÇÃO	5
2 OBJETIVOS E RESULTADOS ESPERADOS	6
3 FUNDAMENTOS TEÓRICOS	7
3.1 Modelos epidemiológicos compartimentais	7
3.1.1 Modelos SIR e SEIR	7
3.1.2 Número básico de reprodução	8
3.2 Modelo baseado em agentes (ABM)	8
3.3 Método de Monte Carlo e geração de números aleatórios	10
3.4 Arquitetura de GPUs	11
3.4.1 Organização dos processadores	11
3.4.2 Hierarquia de memória da GPU	12
3.5 Modelo de programação em CUDA	13
3.6 Métricas de desempenho paralelo	14
4 METODOLOGIA	16
4.1 O Modelo	16
4.2 Inicialização da população	17
4.3 Dinâmica da simulação	17
4.3.1 Mecanismos de contágio	18
4.3.2 Transição entre estados e morte natural	19
4.4 Parâmetros	19
4.5 Estratégia de paralelização e validação	20
5 IMPLEMENTAÇÃO SERIAL	22
5.1 Estrutura de dados	22
5.2 Organização do programa e geração de números aleatórios	23
5.3 Inicialização da população	24
5.4 Laço de simulação e funções de estado	25
5.5 Implementação dos mecanismos de contágio	27
5.6 Atualização síncrona e geração das saídas	29
6 IMPLEMENTAÇÃO EM GPU	31
6.1 Mapeamento da rede em threads	31

6.2	Estrutura de dados na GPU	32
6.2.1	Otimização da estrutura de Health	32
6.3	Geração de números aleatórios thread-safe	33
6.4	Inicialização e laço de simulação	34
6.5	Mecanismos de contágio em paralelo	34
6.6	Atualização e contadores	36
7	RESULTADOS E DISCUSSÃO	38
7.1	Ambiente computacional	38
7.2	Evolução da epidemia nas cidades modeladas	38
7.3	Validação da implementação paralela	40
7.4	Desempenho	42
7.4.1	Aceleração e eficiência	42
7.4.2	Escala com o tamanho da rede	43
7.4.3	Análise do desempenho	44
7.5	Variabilidade dos resultados	47
	REFERÊNCIAS	49

1 Introdução

O novo vírus SARS-CoV-2 que causa a Covid-19, se espalha a partir de pequenas partículas líquidas emitidas por tosse, espirros e saliva de pessoas infectadas. A disseminação da doença está principalmente relacionada ao contato próximo de indivíduos em ambientes confinados, sem ventilação adequada e com interação frequente com um número significativo de pessoas. Pelas características do vírus, a evolução da doença se comporta de maneira diferente por região, elementos específicos de cada área como densidade populacional, pirâmide etária, leitos hospitalares e de tratamento intensivo, são fundamentais para como o espalhamento do vírus e sua letalidade ocorrerá em tal região.

Para tentar prever como uma doença se espalhará em uma epidemia são utilizados modelos epidemiológicos, que podem ser determinísticos, através de equações diferenciais, ou estocásticos, que evoluem através de probabilidades. Neste trabalho será feita uma modelagem baseada em agentes, no qual a evolução da epidemia ocorre através da interação de indivíduos que possuem diversas características como idade, sexo e outras que sejam relevantes ao modelo. A presença de diversos parâmetros e o grande número de agentes para algumas áreas, torna necessário um maior poder computacional para a calibração e implementação do modelo, assim sua paralelização pode ser uma forma eficiente de encontrar os resultados do modelo.

GPUs são orientadas a *throughput* em detrimento de latência e logo conseguem realizar tarefas mais simples e em número maior do que CPUs. Com a popularização de GPUs Nvidia no mercado e a existência da API CUDA (Compute Unified Device Architecture), que estende C/C++ para computação em paralelo. A modelagem baseada em agentes em conjunto com a API CUDA torna-se uma boa tentativa de criar um modelo que possa representar diferentes áreas do Brasil e ainda obter uma eficiência, para que cada área não demande muito tempo.

2 Objetivos e Resultados esperados

Este trabalho possui os seguintes objetivos

- Criação de um modelo baseado em agentes, que contemple as características dos indivíduos e das particularidades das áreas modeladas.
- Mostrar como a Covid-19 impacta uma área de acordo com suas características.
- Implementação do modelo em paralelo utilizando CUDA.
- Validação dos resultados obtidos em paralelo com o obtido em serial.
- Benchmark que mostre uma melhora significativa na eficiência do modelo em paralelo implementado

3 Fundamentos teóricos

3.1 Modelos epidemiológicos compartimentais

3.1.1 Modelos SIR e SEIR

Para descrever a evolução de uma doença infecciosa em uma população ao longo do tempo, utilizam-se os modelos epidemiológicos compartimentais, nos quais a população é dividida em compartimentos que representam os possíveis estados de cada indivíduo em relação à doença, e a dinâmica da epidemia é descrita pela taxa de transição entre esses compartimentos. O modelo mais elementar é o SIR, no qual a população é dividida em três compartimentos: Suscetíveis (S), aqueles que ainda podem contrair a doença; Infecciosos (I), aqueles que possuem a doença e podem transmiti-la; e Removidos (R), aqueles que se recuperaram ou faleceram e não participam mais da transmissão. Em sua formulação determinística, a evolução de cada compartimento ao longo do tempo é descrita por um sistema de equações diferenciais ordinárias.

$$\frac{dS}{dt} = -\beta \frac{SI}{N}, \quad \frac{dI}{dt} = \beta \frac{SI}{N} - \gamma I, \quad \frac{dR}{dt} = \gamma I \quad (3.1)$$

no qual N é o tamanho total da população, β é a taxa de transmissão e γ é a taxa de recuperação, tal que $1/\gamma$ representa o período médio durante o qual o indivíduo permanece infeccioso.

Para doenças nas quais existe um período de latência entre o momento da infecção e o momento em que o indivíduo se torna capaz de transmitir o vírus, o modelo SIR é estendido para o modelo SEIR, no qual é acrescentado o compartimento dos Expostos (E), que representa os indivíduos que já adquiriram a doença, mas ainda passam por um período de incubação e não infectam outros indivíduos.

$$\begin{aligned} \frac{dS}{dt} &= -\beta \frac{SI}{N}, & \frac{dE}{dt} &= \beta \frac{SI}{N} - \sigma E, \\ \frac{dI}{dt} &= \sigma E - \gamma I, & \frac{dR}{dt} &= \gamma I. \end{aligned} \quad (3.2)$$

no qual σ é a taxa de progressão do estado exposto para o estado infeccioso, tal que $1/\sigma$ representa o período médio de latência. A formulação determinística, entretanto, assume uma população homogênea e completamente misturada, na qual todos os indivíduos têm a mesma probabilidade de contato entre si, e logo não captura a estrutura espacial da população, a heterogeneidade entre os indivíduos nem o caráter estocástico do contágio. Para contemplar

essas características, utiliza-se a modelagem baseada em agentes, descrita na Seção 3.2, que pode ser entendida como a contraparte estocástica e espacial dos modelos compartimentais.

3.1.2 Número básico de reprodução

Um dos parâmetros mais importantes na caracterização de uma epidemia é o número básico de reprodução, denotado por R_0 , definido como o número médio de indivíduos que um único caso infecta diretamente em uma população completamente suscetível. O valor de R_0 determina o comportamento inicial da epidemia, uma vez que, quando $R_0 < 1$, cada caso gera em média menos de um novo caso e a doença tende a se extinguir, enquanto, quando $R_0 > 1$, cada caso gera mais de um novo caso e a epidemia se espalha pela população. Nos modelos compartimentais o número básico de reprodução pode ser obtido a partir dos parâmetros da doença; no modelo SIR, por exemplo, $R_0 = \beta/\gamma$, tal que a razão entre a taxa de transmissão e a taxa de recuperação determina o potencial de espalhamento da doença.

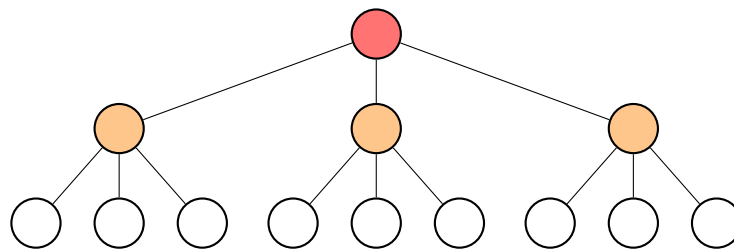


Figura 1 – Ilustração do número básico de reprodução. Cada caso infecta, em média, R_0 novos indivíduos; quando $R_0 > 1$, o número de casos cresce a cada geração.

Como o número básico de reprodução depende da taxa de transmissão β , que por sua vez está relacionada à infecciosidade da doença e à forma de contato entre os indivíduos, a infecciosidade β do modelo é calibrada de modo que o R_0 resultante corresponda ao valor estimado para a Covid-19, adotado neste trabalho como $R_0 = 3,5$. O procedimento de calibração da infecciosidade é descrito no Capítulo 4.

3.2 Modelo baseado em agentes (ABM)

Modelos epidemiológicos abrangem diversos tipos de modelos compartimentais, que incluem uma variedade de características, que simulam o comportamento de doenças infecciosas ao longo de um tempo em uma população. O modelo compartimental utilizado será o SEIR, no qual a população é dividida em quatro compartimentos: Suscetíveis (S), representados por aqueles que ainda não adquiriram a doença e podem adquiri-la; Expostos (E), representados por aqueles que já adquiriram a doença, mas estão passando por um estado de latência e não infectam outros indivíduos; Infecciosos (I), aqueles que possuem a doença e infectam outros indivíduos; e Removidos (R), aqueles que, ou se recuperaram da doença, ou faleceram.

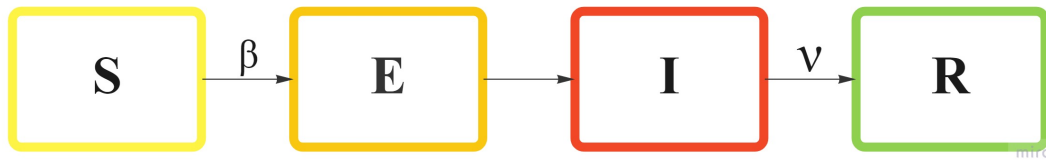


Figura 2 – Fluxograma do funcionamento geral do programa de simulação.

Em ABM, as unidades que interagem entre si, os agentes, apresentam diversas características como estado de saúde, idade, sexo, taxa de isolamento, entre outras que sejam necessárias para a simulação, e são distribuídos de forma heterogênea em uma rede que representa uma população. A evolução da doença na rede depende de como cada agente interage com sua vizinhança e com outros agentes aleatórios. Em uma rede quadrada, o contato entre os agentes pode ser representado das seguintes formas

- Vizinhança de Von Neumann, os 4 vizinhos (norte, leste, sul, oeste).
- Vizinhança de Moore, todos os 8 vizinhos adjacentes ao agente.
- Contatos aleatórios.

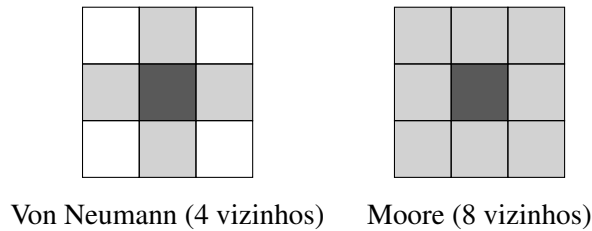


Figura 3 – Vizinhanças em uma rede quadrada. A célula em destaque representa o agente central e as células sombreadas representam os vizinhos considerados no contato: à esquerda, a vizinhança de Von Neumann, com os 4 vizinhos ortogonais; à direita, a vizinhança de Moore, com os 8 vizinhos adjacentes.

Logo, a probabilidade de contágio dependerá da vizinhança e do número de contatos aleatórios, e pode ser representada a partir da seguinte equação.

$$P = 1 - (1 - \beta)^n \quad (3.3)$$

no qual n é a quantidade de infecciosos que tiveram contato com o agente e β é a infecciosidade da doença, que depende de R_0 , parâmetro que representa quantos agentes são infectados a partir de um único caso. Um número aleatório r_n é gerado, e caso $r_n < P$, o agente é infectado.

A dinâmica descrita, na qual cada agente ocupa uma posição fixa em uma rede regular e tem seu estado atualizado a cada passo de tempo em função do estado de sua vizinhança, caracteriza o modelo como um autômato celular probabilístico, no qual as vizinhanças de Von Neumann e de Moore são as mesmas empregadas na teoria de autômatos celulares. Diferentemente de um autômato celular determinístico, no qual a regra de atualização é fixa, no modelo baseado em agentes a transição entre os estados de saúde é governada por probabilidades, e logo cada execução do modelo corresponde a uma realização distinta da epidemia.

3.3 Método de Monte Carlo e geração de números aleatórios

Como a transição dos agentes entre os estados de saúde é decidida de forma probabilística, cada simulação do modelo corresponde a uma única realização do processo estocástico, e logo o resultado de uma simulação isolada não é representativo do comportamento médio da epidemia. Para obter uma estimativa confiável da evolução da doença, utiliza-se o método de Monte Carlo, no qual o modelo é simulado um grande número de vezes de forma independente, e as curvas de prevalência e incidência são obtidas a partir da média sobre todas as simulações realizadas. Quanto maior o número de simulações, menor a flutuação estatística em torno do comportamento médio, uma vez que os desvios de cada realização individual tendem a se compensar.

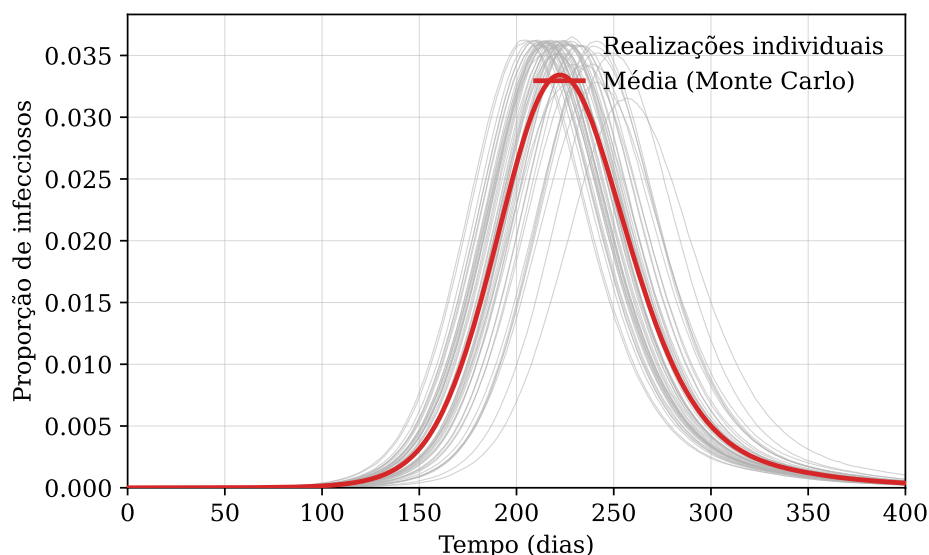


Figura 4 – Realizações individuais do modelo e a média obtida pelo método de Monte Carlo, para a curva de prevalência de infecciosos. Cada realização corresponde a uma execução estocástica do modelo; a média sobre as 50 simulações fornece a estimativa do comportamento epidêmico.

O método de Monte Carlo depende da geração de números aleatórios, entretanto computadores produzem apenas números pseudoaleatórios, gerados de forma determinística a partir de um valor inicial denominado semente, tal que uma mesma semente produz sempre a mesma sequência de números. Em uma implementação serial uma única sequência de números pseudoaleatórios é suficiente, porém em uma implementação paralela a função geradora precisa ser *thread-safe*, uma vez que diversas threads acessam o gerador simultaneamente, e a utilização de uma mesma sequência por todas as threads introduziria correlações indesejadas entre os agentes. Para contornar essa limitação, CUDA disponibiliza a biblioteca cuRAND, que oferece geradores de números pseudoaleatórios apropriados para a execução em paralelo.

3.4 Arquitetura de GPUs

3.4.1 Organização dos processadores

Enquanto CPUs são orientadas para realizar operações mais complexas, mas com baixa latência, GPUs são orientadas para maior *throughput*, realizam operações mais simples, mas processam bastantes tarefas ao mesmo tempo. Essas características tornam CPUs excelentes para tarefas em serial, enquanto GPUs são ótimas para tarefas paralelizáveis. Essa diferença ocorre devido a suas arquiteturas, CPUs têm poucos *cores* que precisam de memória cache para diminuir a latência, enquanto GPUs não se preocupam tanto com latência, então possuem menos transistores para memória cache em benefício de mais transistores que realizarão processamento de tarefas, dependendo da família da GPU, centenas e até milhares de *cores* estão presentes.

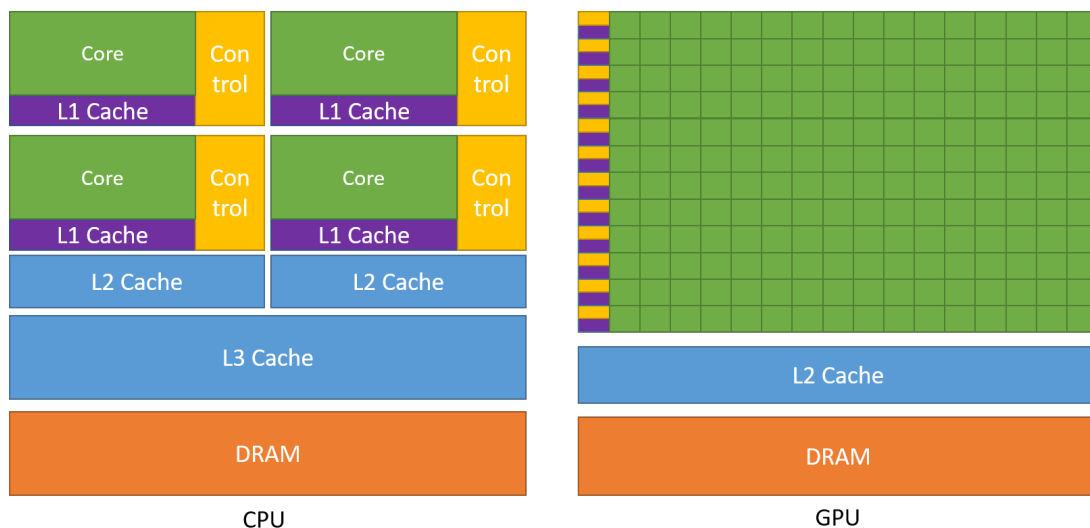


Figura 5 – Arquitetura de uma CPU e de uma GPU.

GPUs contêm SMs (*streaming multiprocessors*), que contêm CUDA *cores* que realizarão as tarefas, cache L1, memória compartilhada entre os processadores dentro do SM e cache L2.

Para obter maior *throughput*, os *cores* nos SMs são SIMT (*single instruction multiple thread*), para cada tarefa necessária, um grupo de 32 *cores* realizará a mesma tarefa, esses grupos são chamados de *warps*, e o processamento das tarefas será sempre realizado pela execução dos *warps*.

3.4.2 Hierarquia de memória da GPU

Além da grande quantidade de *cores*, o desempenho de um programa executado em GPU depende fortemente da forma como os dados são acessados na memória, organizada em uma hierarquia de níveis com diferentes capacidades e latências. No nível mais próximo dos *cores* estão os registradores, privados de cada thread e de acesso mais rápido; em seguida está a memória compartilhada, acessível por todas as threads de um mesmo bloco e localizada dentro do SM, de baixa latência; e, no nível mais distante, está a memória global, de maior capacidade, porém de maior latência, na qual residem os dados copiados do *host* para o *device*. Entre a memória global e os SMs encontram-se ainda as memórias cache L1, privada de cada SM, e L2, compartilhada entre todos os SMs, que armazenam os dados acessados recentemente com o intuito de reduzir o número de acessos à memória global.

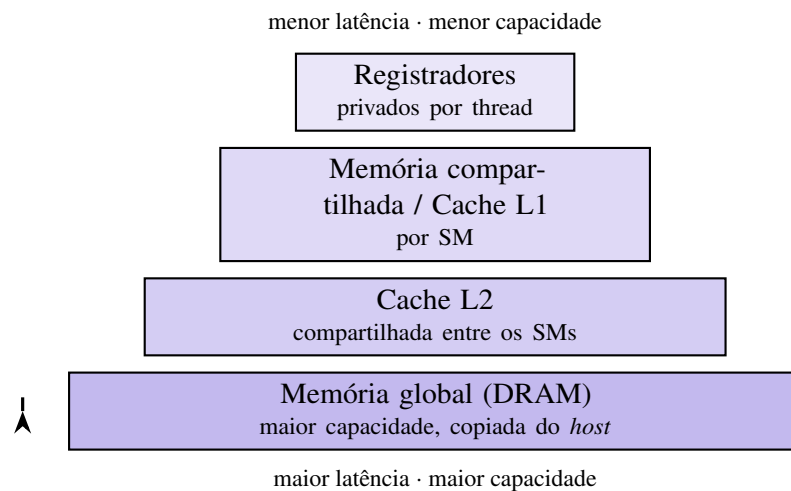


Figura 6 – Hierarquia de memória de uma GPU. Do topo para a base, a capacidade e a latência de acesso aumentam, enquanto a proximidade em relação aos *cores* diminui.

O acesso à memória global é mais eficiente quando é coalescido, ou seja, quando threads de uma mesma warp acessam posições contíguas de memória, que podem então ser atendidas em uma única transação. Em diversas aplicações o desempenho não é limitado pela capacidade de processamento, mas pela largura de banda da memória, situação na qual o programa é dito limitado por memória (*memory-bound*), e logo a forma como os dados são estruturados e acessados torna-se determinante para o desempenho. Em particular, estruturas de dados compactas, que reduzem o volume de dados transferidos e permitem que o conjunto de

dados em uso seja mantido na memória cache L2, podem proporcionar ganhos expressivos de desempenho.

3.5 Modelo de programação em CUDA

CUDA é uma API (*Application Programming Interface*) que permite a criação de programas em C/C++ em GPUs Nvidia que sejam compatíveis. Em CUDA, o programador cria funções semelhantes às funções de C/C++; essas funções, chamadas de kernels, realizam N tarefas para as N CUDA threads disponíveis na GPU utilizada. Um conjunto de threads é chamado de "blocks" e um conjunto de "blocks" forma um "grid", esses conjuntos podem ter até 3 dimensões, e podem ser identificados com variáveis nativas dadas por: **threadIdx**, **blockIdx**, **blockDim** e **gridDim**. Manipulando essas variáveis, o programador consegue acessar e realizar tarefas para todas as threads.

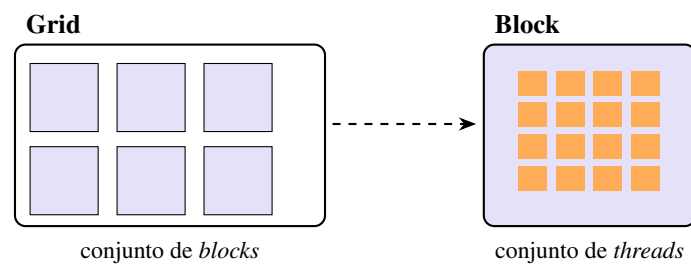


Figura 7 – Hierarquia de execução em CUDA: as threads são agrupadas em *blocks* e os *blocks* formam um *grid*. À direita, a ampliação de um *block* e suas threads.

Ao desenvolver programas utilizando a arquitetura CUDA, está-se envolvido em uma computação heterogênea, uma vez que o programa faz uso da interação entre a unidade central de processamento (CPU), também conhecida como *host*, e a unidade de processamento gráfico (GPU), denominada *device*. No *host*, o código é responsável por realizar as tarefas que precisam ser feitas em serial, alocações de memória, copiar os dados da memória da CPU para a GPU e então chamar os kernels para realizar as tarefas no *device*. Na GPU, cada SM realizará as tarefas por bloco e, após a finalização das execuções, o *host* copia os resultados obtidos da memória da GPU para a CPU. A Figura 8 representa como funciona um programa feito em CUDA.

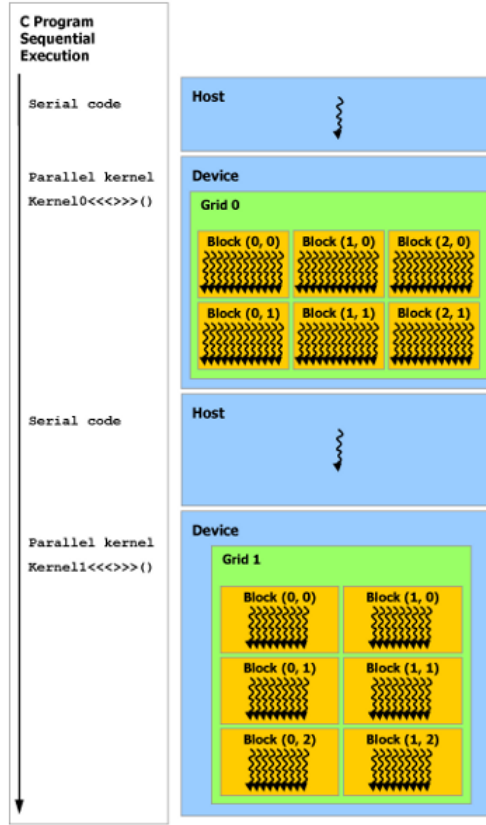


Figura 8 – Execução de um programa feito em CUDA.

3.6 Métricas de desempenho paralelo

Para avaliar o ganho obtido com a paralelização de um programa, utilizam-se métricas que comparam o desempenho da implementação paralela com o da implementação serial. A principal delas é a aceleração (*speedup*), definida como a razão entre o tempo de execução da implementação serial e o tempo de execução da implementação paralela.

$$S = \frac{T_{serial}}{T_{paralelo}} \quad (3.4)$$

no qual T_{serial} é o tempo de execução do programa em um único processador e $T_{paralelo}$ é o tempo de execução do programa paralelizado. Uma segunda métrica é a eficiência, definida como a razão entre a aceleração obtida e o número de unidades de processamento utilizadas.

$$E = \frac{S}{p} \quad (3.5)$$

no qual p é o número de unidades de processamento, tal que a eficiência indica o quanto cada unidade contribui, em média, para a aceleração total.

A aceleração que pode ser obtida, entretanto, é limitada pela fração do programa que precisa ser executada de forma serial, relação expressa pela Lei de Amdahl.

$$S = \frac{1}{f + \frac{1-f}{p}} \quad (3.6)$$

no qual f é a fração do programa que permanece serial, tal que, à medida que o número de unidades de processamento p aumenta, a aceleração tende ao limite $1/f$, e logo a parcela serial do programa determina o ganho máximo que pode ser alcançado, independentemente da quantidade de *cores* disponíveis. Em GPUs, nas quais milhares de threads são executadas simultaneamente, a aceleração é frequentemente medida de forma empírica, comparando-se diretamente o tempo de execução da implementação paralela com o da implementação serial de referência.

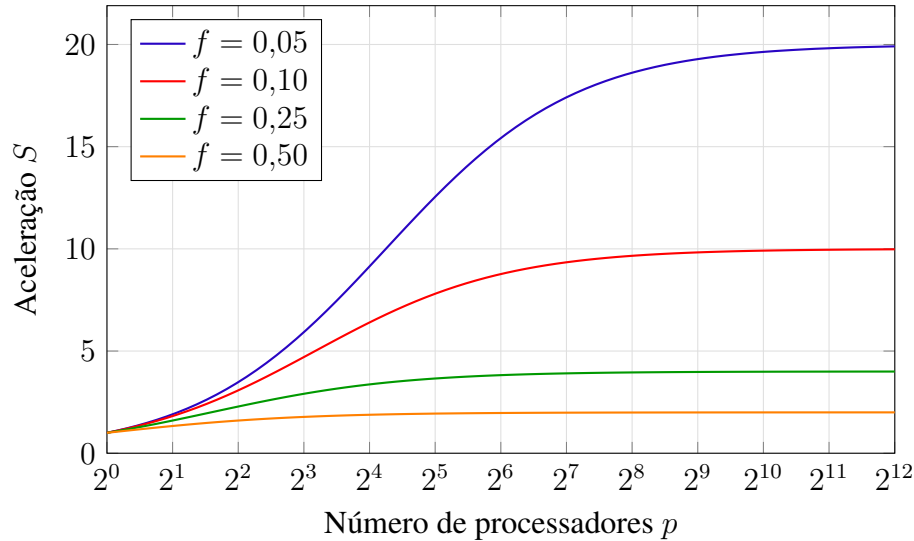


Figura 9 – Lei de Amdahl: aceleração S em função do número de processadores p para diferentes frações seriais f . A aceleração satura no limite $1/f$, evidenciando que a parcela serial do programa limita o ganho máximo, independentemente do número de processadores.

4 Metodologia

4.1 O Modelo

A modelagem baseada em agentes utilizada será uma modificação do modelo compartimental SEIR; os indivíduos podem ter os seguintes estados de saúde: suscetíveis (X), expostos (E), pré-sintomáticos (P), assintomáticos (A), sintomático leve (S_l), sintomático moderado (S_m), sintomático severo (S_s), hospitalizado (H), internados na UTI (UTI), recuperados (R) e mortos (M). A Figura 10 abaixo demonstra como ocorrem os possíveis movimentos dentre os estados de saúde.

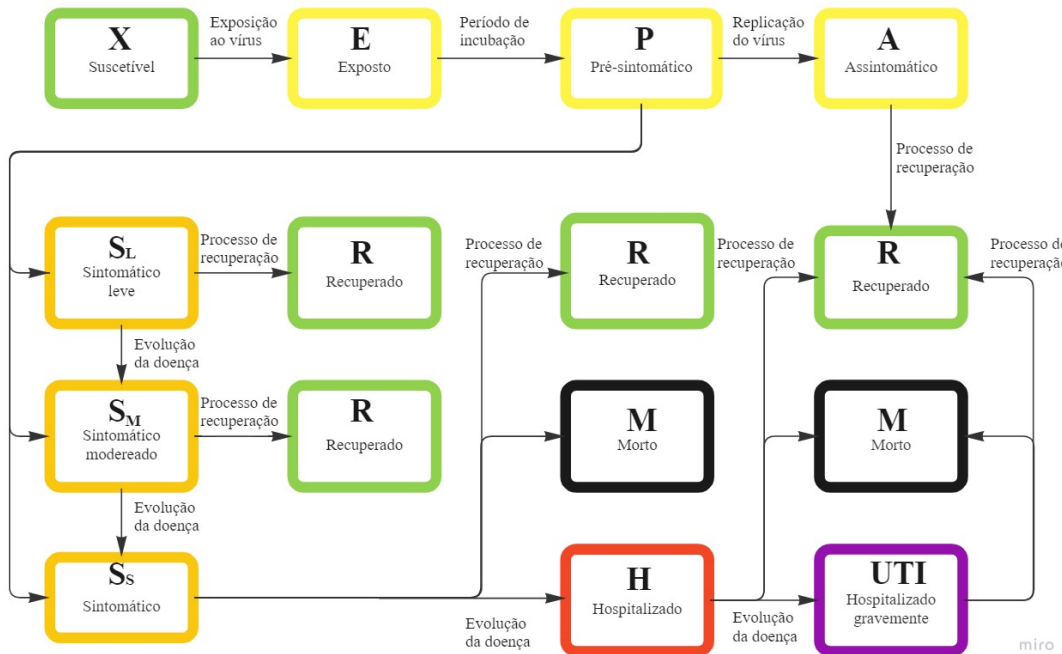


Figura 10 – Fluxograma do movimento entre estados de saúde.

O indivíduo suscetível (X) entra em contato com o vírus e torna-se exposto (E), após o período de incubação torna-se pré-sintomático (P), a replicação do vírus ocorre e o indivíduo ou torna-se assintomático (A) e se recupera da doença, ou a doença evolui e o indivíduo se torna infeccioso, o indivíduo então pode ter sintomas leves (S_l), moderados (S_m) ou severos (S_s), caso não se recupere, será hospitalizado (H) e, em casos mais graves, internado em unidades de tratamento intensivo (UTI); na UTI o indivíduo ou se recupera, ou morre devido à doença.

No modelo, cada indivíduo ocupa uma posição em uma matriz quadrada de lado L com população total $N = L \times L$, tal que N é referente a cada cidade analisada. No início de cada simulação a população é iniciada com todos os indivíduos com estado suscetível (X),

exceto 5 indivíduos que estarão em estado pré-sintomático, os indivíduos infectados entrarão em contato com sua vizinhança, Von Neumann para as cidades de baixa densidade populacional ou Moore para cidades de alta densidade, além dos contatos com outros indivíduos aleatórios. A probabilidade de infecção dos indivíduos suscetíveis é dada pela equação 3.3, um número aleatório r é gerado e caso $r < P$, o indivíduo é infectado.

4.2 Inicialização da população

No início de cada simulação, além de receber o estado de saúde, cada indivíduo recebe uma idade atribuída de acordo com a estrutura etária da população brasileira, de modo que a distribuição de idades da rede reproduza a pirâmide etária real do país. A cada indivíduo é também atribuída uma idade de morte natural, sorteada por amostragem por rejeição a partir das probabilidades de morte natural por faixa etária, tal que a idade de morte respeita a expectativa de vida observada. Dessa forma, a rede inicia com uma população heterogênea em idade, o que é relevante uma vez que a evolução da Covid-19 e a sua letalidade dependem fortemente da faixa etária do indivíduo.

Os leitos de hospital e de UTI disponíveis para os doentes de Covid-19 são definidos a partir da razão de leitos por habitante de cada cidade, descontada a ocupação inicial de 50% por outras doenças, de modo que apenas metade dos leitos esteja efetivamente disponível no início da simulação. A epidemia é iniciada com 5 indivíduos no estado pré-sintomático (P), distribuídos aleatoriamente na rede, enquanto os demais indivíduos permanecem suscetíveis.

4.3 Dinâmica da simulação

A simulação evolui em passos de tempo discretos, nos quais cada passo corresponde a um dia. A cada passo, antes de atualizar os estados, são aplicadas condições de contorno periódicas à rede, de modo que os indivíduos das bordas tenham como vizinhos os indivíduos da borda oposta, e logo a rede se comporta como uma superfície sem fronteiras, evitando efeitos de borda. Em seguida, percorre-se toda a rede e, para cada indivíduo, é aplicada a regra de transição correspondente ao seu estado de saúde atual, que determina o seu próximo estado. Por fim, a rede é atualizada de forma síncrona, ou seja, as transições calculadas para todos os indivíduos são aplicadas simultaneamente ao final do passo, de modo que a atualização de um indivíduo não interfira no cálculo dos demais no mesmo passo. O Algoritmo 1 resume esse procedimento.

```

for cada simulação do
  inicializa a população: estados de saúde, idades e leitos disponíveis
  for cada passo de tempo  $t$  correspondente a um dia do
    aplica as condições de contorno periódicas à rede
    foreach indivíduo da rede do
      aplica a regra de transição correspondente ao seu estado de saúde
    end
    atualiza a rede de forma síncrona
    substitui os indivíduos que sofreram morte natural por novos suscetíveis
    contabiliza a prevalência e a incidência de cada estado
  end
end

calcula a média das curvas sobre todas as simulações

```

Algoritmo 1: Laço principal da simulação

4.3.1 Mecanismos de contágio

A transmissão da doença ocorre por dois mecanismos complementares, ambos baseados na probabilidade de contágio dada pela Equação 3.3. No primeiro mecanismo, cada indivíduo suscetível verifica a sua vizinhança e os seus contatos aleatórios, cujo número varia conforme a cidade, em busca de indivíduos infectados; o número de infectados encontrados, n , determina a probabilidade $P = 1 - (1 - \beta)^n$ de o suscetível ser infectado. No segundo mecanismo, cada indivíduo infectado, enquanto se encontra no período infeccioso, procura indivíduos suscetíveis entre seus contatos e tenta infectá-los, também de acordo com a probabilidade de contágio.

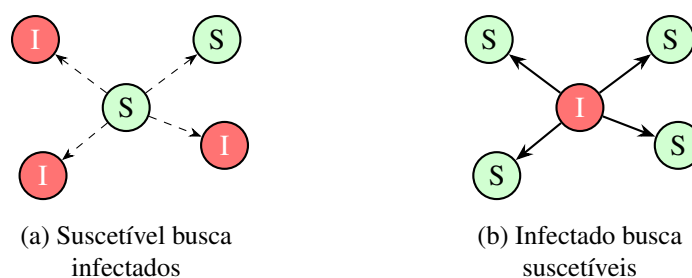


Figura 11 – Os dois mecanismos de contágio do modelo. Em (a), o indivíduo suscetível verifica a sua vizinhança e os seus contatos aleatórios em busca de infectados; em (b), o indivíduo infectado procura indivíduos suscetíveis entre seus contatos e tenta infectá-los. As setas indicam o sentido da busca em cada mecanismo.

O segundo mecanismo é executado pelos estados pré-sintomático (P), assintomático (A) e sintomático leve (S_l), que representam os indivíduos infecciosos que ainda circulam na população, enquanto os estados sintomático moderado (S_m) e severo (S_s), por estarem afastados do convívio social, não procuram novos suscetíveis. Para evitar que um mesmo suscetível seja

infectado duas vezes no mesmo passo de tempo, uma vez pela sua própria busca e outra por um infectado que o encontrou, utiliza-se um mecanismo de marcação que verifica se o indivíduo já foi contabilizado, garantindo a consistência da atualização.

4.3.2 Transição entre estados e morte natural

Cada estado de saúde possui uma duração, sorteada no momento em que o indivíduo entra no estado, dentro dos limites mínimo e máximo definidos para aquele estado (Tabela 2). Enquanto a duração não é atingida, o indivíduo permanece no mesmo estado; ao atingi-la, ocorre a transição para o próximo estado, que pode ser determinística ou sorteada de acordo com as probabilidades de transição (Tabela 3) e, em alguns casos, com a probabilidade de recuperação correspondente à faixa etária do indivíduo. A passagem para os estados de hospitalização (H) e de UTI depende ainda da disponibilidade de leitos, de modo que um indivíduo em estado severo só é hospitalizado se houver leito disponível.

Independentemente do estado de saúde, qualquer indivíduo pode sofrer morte natural ao atingir a sua idade de morte, situação na qual é removido e imediatamente substituído por um novo indivíduo suscetível, com idade sorteada novamente a partir da estrutura etária, de modo que o tamanho total da população permaneça constante ao longo da simulação.

4.4 Parâmetros

Para cada cidade a ser utilizada no modelo, a calibração da infecciosidade β é necessária para que o valor de $R_0 = 3,5$ seja padrão em todas as cidades. É necessário também alterar os parâmetros que são específicos de cada cidade, são eles: tamanho da população, densidade populacional, contatos aleatórios, leitos de hospital e leitos de UTI. A Tabela 1 mostra as características de 4 cidades: São Paulo (SP), Manaus (AM), Brasília (DF) e a favela da Rocinha (RJ).

Área	Densidade populacional	L	Leitos/Pop	Leitos de UTI/Pop	Contatos aleatórios
São Paulo	Alta	3355	0,00247452	0,00043782	2-19
Rocinha	Alta	264	0,00055111	0,00014592	2-120
Brasília	Baixa	1604	0,00260879	0,00040114	2
Manaus	Baixa	1343	0,00187124	0,00027858	2

Tabela 1 – Parâmetros de 4 diferentes áreas do Brasil.

Já os parâmetros relacionados à doença são comuns a todas as áreas modeladas. A Tabela 2 mostra a duração, em dias, o período de cada estado de saúde, a Tabela 3 mostra a probabilidade de transição entre os estados de saúde.

Parâmetro	Descrição	Duração (dias)
τ_l	Período de latência	0 - 27
τ_P	Pré-sintomático	0 - 14
τ_A	Assintomático	0 - 7
τ_{S_l}	Sintomático leve	0 - 14
τ_{S_m}	Sintomático moderado	0 - 28
τ_{S_s}	Sintomático severo	0 - 4
τ_H	Hospitalização	7 - 45
τ_{UTI}	UTI	10 - 60

Tabela 2 – Parâmetros de duração de cada estado de saúde.

Parâmetro	Transição entre estados	Probabilidade
$P_{(P \rightarrow A)}$	Pré-sintomático para assintomático	0.5
$P_{(P \rightarrow S_l)}$	Pré-sintomático para sintomático leve	0.6
$P_{(P \rightarrow S_m)}$	Pré-sintomático para sintomático moderado	0.2
$P_{(P \rightarrow S_s)}$	Pré-sintomático para sintomático severo	0.2
$P_{(S_l \rightarrow S_m)}$	Sintomático leve para sintomático moderado	0.1
$P_{(S_s \rightarrow H)}$	Sintomático severo para hospitalização	0.75
$P_{(S_s \rightarrow UTI)}$	Sintomático severo para internação em UTI	0.25

Tabela 3 – Probabilidades de transição entre estados de saúde.

4.5 Estratégia de paralelização e validação

A estratégia de paralelização adotada baseia-se no paralelismo de dados: uma vez que, a cada passo de tempo, todos os indivíduos da rede são processados de forma independente, associa-se uma thread a cada indivíduo, de modo que toda a rede é processada simultaneamente. A dinâmica da simulação é decomposta em kernels, um para cada estado de saúde, lançados em sequência a cada passo; como o lançamento de cada kernel funciona como um ponto de sincronização global, a ordem em que os estados são tratados na implementação serial é preservada. A geração de números aleatórios, por sua vez, é adaptada para a execução paralela, atribuindo-se um estado de gerador a cada thread. Em todas essas decisões, o princípio orientador é a preservação do comportamento do modelo, de modo que a implementação paralela reproduza os resultados da implementação serial.

Para que a paralelização seja considerada bem-sucedida, não basta que seja mais rápida; é necessário que produza os mesmos resultados da implementação serial, tomada como referência. A validação é feita comparando-se as curvas epidêmicas obtidas pelas duas implementações sob os mesmos parâmetros — mesma infecciosidade β , mesmo tamanho de rede e mesmo número de simulações —, para cada uma das cidades modeladas. A concordância entre as curvas é quantificada pelo erro quadrático médio entre as proporções de cada estado ao longo do tempo, de modo que uma diferença pequena, compatível com a flutuação estatística do método de Monte Carlo, indica a equivalência entre as implementações.

Confirmada a equivalência, avalia-se o ganho de desempenho da implementação paralela. A avaliação é feita medindo-se a aceleração, definida no Capítulo 3 como a razão entre o tempo de execução da implementação serial e o da implementação paralela, e a eficiência correspondente, para cada cidade. Como as cidades possuem tamanhos de rede distintos, essa avaliação permite ainda observar como o ganho de desempenho varia com o tamanho do problema. A Figura 12 resume a estratégia do trabalho.

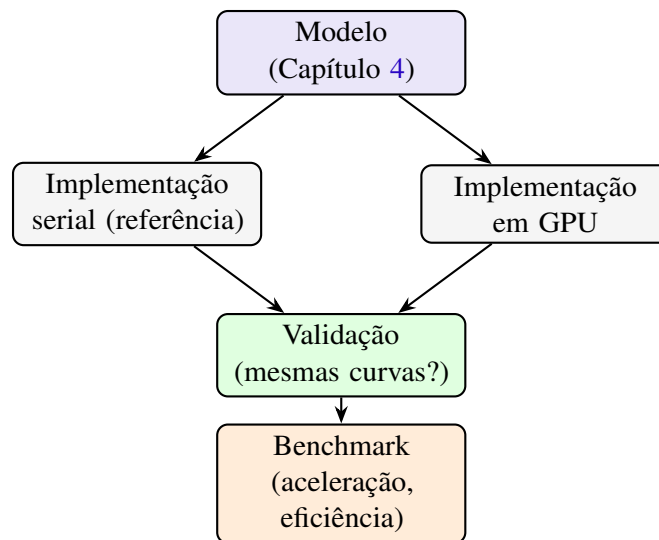


Figura 12 – Estratégia do trabalho: o mesmo modelo é realizado em duas implementações; a implementação em GPU é validada contra a serial sob os mesmos parâmetros e, confirmada a equivalência, avalia-se o seu desempenho.

5 Implementação serial

A implementação serial do modelo, escrita na linguagem C, constitui a referência deste trabalho, uma vez que é contra os seus resultados que a implementação paralela será validada. Neste capítulo descreve-se como o modelo apresentado no Capítulo 4 é realizado como um programa sequencial, no qual um único processador percorre toda a rede a cada passo de tempo.

5.1 Estrutura de dados

Cada indivíduo da população é representado por uma estrutura que reúne o seu estado de saúde e os atributos necessários à evolução da doença, e a rede é representada por uma matriz dessas estruturas. A Listagem 5.1 apresenta os principais campos dessa estrutura.

```
struct Individual {  
    int Health;           // estado de saúde atual  
    int Swap;             // próximo estado (atualização síncrona)  
    int AgeYears;         // idade em anos  
    int AgeDays;          // idade em dias  
    int AgeDeathYears;    // idade de morte natural (anos)  
    int AgeDeathDays;     // idade de morte natural (dias)  
    int StateTime;        // duração sorteada para o estado atual  
    int TimeOnState;      // tempo já decorrido no estado atual  
    int Days;             // dias do indivíduo na simulação  
    int Isolation;        // indicador de isolamento  
    int Exponent;         // tentativas de infecção no passo  
    int Checked;         // marca para evitar dupla infecção  
} Person[L+2][L+2];
```

Listing 5.1 – Estrutura que representa cada indivíduo (agente).

O campo `Health` armazena o estado de saúde atual do indivíduo, enquanto o campo `Swap` armazena o estado para o qual ele transitará, mecanismo detalhado adiante. Os campos de idade guardam a idade atual e a idade de morte natural do indivíduo, e os campos `StateTime` e `TimeOnState` guardam, respectivamente, a duração sorteada para o estado atual e o tempo já decorrido nesse estado, de modo que a transição ocorre quando `TimeOnState` atinge `StateTime`. Por fim, os campos `Exponent` e `Checked` são utilizados no controle do contágio, evitando que um mesmo suscetível seja infectado mais de uma vez no mesmo passo.

A rede é declarada como uma matriz `Person[L+2][L+2]`, na qual as duas linhas e

as duas colunas adicionais formam uma borda que armazena cópias das extremidades opostas da rede, implementando as condições de contorno periódicas descritas no Capítulo 4; dessa forma, ao verificar a vizinhança de um indivíduo da borda, o programa encontra automaticamente os vizinhos da borda oposta.

A atualização da rede é feita de forma síncrona por meio de um duplo-buffer, no qual os campos `Health` e `Swap` desempenham papéis complementares. Durante a varredura da rede, as funções de estado leem o estado atual de cada indivíduo em `Health` e escrevem o seu próximo estado em `Swap`, sem alterar `Health`; somente ao final do passo o conteúdo de `Swap` é copiado para `Health`. Assim, garante-se que a atualização de um indivíduo não interfira no cálculo dos demais no mesmo passo, conforme ilustra a Figura 13.

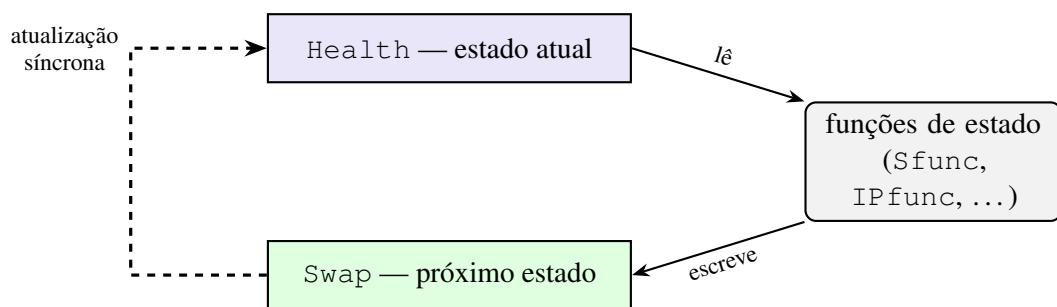


Figura 13 – Duplo-buffer da atualização síncrona. Durante a varredura, as funções de estado leem o estado atual em `Health` e escrevem o próximo estado em `Swap`; ao final do passo, `Swap` é copiado para `Health`, de modo que todas as transições de um passo são aplicadas simultaneamente.

5.2 Organização do programa e geração de números aleatórios

O programa é organizado de forma modular, no qual o arquivo principal contém o laço de simulação e inclui diversos arquivos auxiliares, cada um responsável por uma parte do modelo: a inicialização da população, os parâmetros de cada cidade, as probabilidades de morte natural e de recuperação por faixa etária, os dois mecanismos de contágio e as funções de estado. Cada estado de saúde possui a sua própria função, o que mantém o código legível e preserva a correspondência direta entre o modelo e a sua implementação.

Como o modelo é estocástico, a geração de números aleatórios é parte essencial da implementação. Utiliza-se um gerador congruencial linear multiplicativo, apresentado na Listagem 5.2, no qual o estado do gerador é multiplicado por uma constante a cada chamada e normalizado para o intervalo $[0, 1]$. A cada simulação o gerador é inicializado com uma semente distinta, derivada do número da simulação, de modo que cada simulação produz uma sequência própria de números pseudoaleatórios e, ao mesmo tempo, os resultados permanecem reprodutíveis.

```

double aleat() {
    R *= mult;                // mult = 888121
    rn = (double) R / MAXNUM; // número em [0, 1]
}

```

Listing 5.2 – Gerador congruencial linear multiplicativo.

5.3 Inicialização da população

Antes do início do laço de simulação, a função de inicialização prepara a população de acordo com o descrito no Capítulo 4. Toda a rede é colocada no estado suscetível e, a cada indivíduo, é atribuída uma idade sorteada a partir da estrutura etária brasileira, bem como uma idade de morte natural obtida por amostragem por rejeição sobre as probabilidades de morte por faixa etária. Em seguida, os 5 casos iniciais são posicionados aleatoriamente na rede no estado pré-sintomático. A Listagem 5.3 apresenta a estrutura dessa função.

```

void beginfunc() {
    for (i = 1; i <= L; i++)
        for (j = 1; j <= L; j++) {
            Person[i][j].Health = S;                // toda a rede em
                                                    suscetível

            // sorteia a faixa etária pela estrutura etária e a idade
            na faixa
            Person[i][j].AgeYears = rn * (MaximumAge - MinimumAge) +
                MinimumAge;
            aleat();
            Person[i][j].AgeDays = rn * 365;

            // idade de morte natural por amostragem por rejeição
            do {
                aleat(); Person[i][j].AgeDeathYears = rn * 100;
                aleat();
            } while (rn >= ProbNaturalDeath[Person[i][j].AgeDeathYears
                ]);
            Person[i][j].AgeDeathDays = Person[i][j].AgeDeathYears *
                365;
        }

    // distribui os IPini = 5 casos iniciais em posições aleatórias
}

```



```

for (k = 0; k < IPini; ) {
    aleat(); i = rn * L + 1;
    aleat(); j = rn * L + 1;
    if (Person[i][j].Health == S) {
        Person[i][j].Health = IP;
        aleat();
        Person[i][j].StateTime = rn * (MaxIP - MinIP) + MinIP;
        k++;
    }
}

```

Listing 5.3 – Inicialização da população (beginfunc).

5.4 Laço de simulação e funções de estado

O laço principal segue o procedimento descrito no Algoritmo 1. Para cada simulação, a população é inicializada e, em seguida, a simulação avança dia a dia: a cada passo, as condições de contorno periódicas são aplicadas e a rede é percorrida sequencialmente, de modo que, para cada indivíduo, é chamada a função correspondente ao seu estado de saúde atual. Concluída a varredura, a rede é atualizada. A Listagem 5.4 apresenta esse laço, no qual `estado_IS(·)` abrevia o teste dos estados sintomáticos (IA , S_l , S_m e S_s).

```

for (sim_time = 0; sim_time <= DAYS; sim_time++) {
    // condições de contorno periódicas (bordas e cantos)
    for (i = 1; i <= L; i++) {
        Person[0][i].Health = Person[L][i].Health;
        Person[L+1][i].Health = Person[1][i].Health;
        Person[i][0].Health = Person[i][L].Health;
        Person[i][L+1].Health = Person[i][1].Health;
    }

    // varredura da rede: despacho por estado de saúde
    for (i = 1; i <= L; i++)
        for (j = 1; j <= L; j++)
            if (Person[i][j].Health == S) Sfunc(i, j);
            else if (Person[i][j].Health == E) Efunc(i, j);
            else if (Person[i][j].Health == IP) IPfunc(i, j);
            else if (estado_IS(Person[i][j].Health)) ISfunc(i, j);
            else if (Person[i][j].Health == H) Hfunc(i, j);
            else if (Person[i][j].Health == ICU) ICUfunc(i, j);

```

```

Updatefunc();    // atualização síncrona
}

```

Listing 5.4 – Laço principal: contorno periódico, varredura e atualização.

Cada função de estado segue uma estrutura semelhante: incrementa o tempo decorrido no estado, verifica se o indivíduo sofreu morte natural e, caso contrário, decide a sua permanência ou transição. Enquanto o tempo decorrido não atinge a duração sorteada, o indivíduo permanece no mesmo estado; ao atingi-la, a função sorteia o próximo estado de acordo com as probabilidades de transição e, quando aplicável, com a probabilidade de recuperação correspondente à sua faixa etária, escrevendo o resultado no campo Swap. As funções dos estados infecciosos que ainda circulam na população acrescentam, enquanto o indivíduo permanece infeccioso, a chamada ao segundo mecanismo de contágio, descrito a seguir. A Listagem 5.5 ilustra essa estrutura para o estado pré-sintomático.

```

void IPfunc(int i, int j) {
    Person[i][j].TimeOnState++;

    if (Person[i][j].Days >= Person[i][j].AgeDeathDays) {    //
        morte natural
        Person[i][j].Swap = Dead;
        New_Dead++;
    }
    else if (Person[i][j].TimeOnState >= Person[i][j].StateTime) {
        // duração do estado terminou: sorteia o próximo estado
        aleat();
        if (rn < ProbIPtoIA) {
            Person[i][j].Swap = IA;
            aleat();
            Person[i][j].StateTime = rn * (MaxIA - MinIA) + MinIA;
        } else {
            // sorteia ISLight / ISModerate / ISSevere conforme as
            // probabilidades
            // ...
        }
    }
    else {
        // ainda infeccioso
        Neighborsinfectedfunc(i, j); // segundo mecanismo de contá
        gio
        Person[i][j].Swap = IP;
    }
}

```

```
}
```

Listing 5.5 – Função de estado do pré-sintomático (IPfunc).

As funções dos estados sintomático severo, hospitalizado e de UTI consideram ainda a disponibilidade de leitos: um indivíduo em estado severo só é hospitalizado se houver leito de hospital disponível, e a internação em UTI depende da disponibilidade de leitos de UTI. O número de leitos disponíveis em cada cidade desconta a ocupação inicial por outras doenças, conforme descrito no Capítulo 4, e é controlado por contadores atualizados ao longo da simulação.

5.5 Implementação dos mecanismos de contágio

O primeiro mecanismo de contágio é realizado pela função associada ao estado suscetível, que percorre a vizinhança local do indivíduo — de Moore para as cidades de alta densidade e de Von Neumann para as de baixa densidade — e os seus contatos aleatórios, contando o número n de indivíduos infectados encontrados. A partir desse número, calcula-se a probabilidade de contágio $P = 1 - (1 - \beta)^n$ e, com um número aleatório, decide-se se o suscetível é infectado, caso em que ele transita para o estado exposto. A Listagem 5.6 apresenta esse mecanismo, no qual `infeccioso(·)` denota o teste de o indivíduo estar em um estado infeccioso.

```
int Neighborsfunc(int i, int j) {
    int KI = 0;           // número de infectados encontrados
    Contagion = 0;

    if (Person[i][j].Isolation == IsolationNo) { // contatos
        aleatórios
        aleat();
        RandomContacts = rn * (MaxRandomContacts -
                               MinRandomContacts) + MinRandomContacts;
        for (mute = 0; mute < RandomContacts; mute++) {
            // sorteia um contato (Randomi, Randomj)
            if (infeccioso(Randomi, Randomj)) KI++;
        }
    }

    #if (Density == HIGH) // vizinhança de Moore (8
        vizinhos)
        for (k = -1; k <= 1; k++)
            for (l = -1; l <= 1; l++)
                if (infeccioso(i + k, j + l)) KI++;
    #else // vizinhança de Von Neumann (4
```

```

vizinhos)
    if (infeccioso(i-1,j) || infeccioso(i+1,j) ||
        infeccioso(i,j-1) || infeccioso(i,j+1)) KI++;
#endif

    if (KI > 0) {
        // P = 1 - (1 - Beta)^KI
        ProbContagion = 1.0 - pow(1.0 - Beta, (double) KI);
        aleat();
        if (rn <= ProbContagion) Contagion = 1;
    }
    return Contagion;
}

```

Listing 5.6 – Primeiro mecanismo: o suscetível busca infectados (Neighborsfunc).

O segundo mecanismo é realizado pelos indivíduos infecciosos que ainda circulam na população, que procuram indivíduos suscetíveis entre seus contatos aleatórios e tentam infectá-los. Para garantir a consistência com a atualização síncrona e evitar que um mesmo suscetível seja infectado duas vezes no mesmo passo, cada suscetível dispõe dos campos *Exponent* e *Checked*: o primeiro contabiliza as tentativas de infecção dirigidas àquele indivíduo no passo, enquanto o segundo marca o indivíduo já processado, de modo que a infecção é aplicada uma única vez por passo, independentemente de ter origem na busca do próprio suscetível ou na de um infectado. A Listagem 5.7 apresenta esse mecanismo.

```

void Neighborsinfectedfunc(int i, int j) {
    if (Person[i][j].Isolation == IsolationNo) {
        aleat();
        RandomContacts = rn * (MaxRandomContacts -
            MinRandomContacts) + MinRandomContacts;
        for (mute = 0; mute < RandomContacts; mute++) {
            // sorteia um suscetível-alvo (Randomi, Randomj)
            if (Person[Randomi][Randomj].Health == S)
                Person[Randomi][Randomj].Exponent++;

            if (Person[Randomi][Randomj].Checked == 0 &&
                Person[Randomi][Randomj].Exponent == 1) {
                ProbContagion = 1.0 - pow(1.0 - Beta, (double)
                    Person[Randomi][Randomj].Exponent);
                aleat();
                if (rn <= ProbContagion) {
                    Person[Randomi][Randomj].Checked = 1;
                    Person[Randomi][Randomj].Swap = E;
                }
            }
        }
    }
}

```

```

        Person[Randomi][Randomj].TimeOnState = 0;
        aleat();
        Person[Randomi][Randomj].StateTime = rn * (
            MaxLatency - MinLatency) + MinLatency;
    }
}
}
}
}

```

Listing 5.7 – Segundo mecanismo: o infectado busca suscetíveis (Neighborsinfectedfunc).

5.6 Atualização síncrona e geração das saídas

Ao final de cada passo de tempo, a função de atualização copia o campo `Swap` para o campo `Health` de todos os indivíduos, efetivando de forma síncrona as transições calculadas durante a varredura, e incrementa os contadores de idade e de tempo de simulação. Em seguida, contabiliza-se quantos indivíduos se encontram em cada estado de saúde, e os indivíduos que atingiram a sua idade de morte natural são removidos e substituídos por novos suscetíveis, com idade sorteada novamente a partir da estrutura etária, de modo que o tamanho total da população permanece constante. A Listagem 5.8 apresenta o trecho central dessa atualização.

```

for (i = 1; i <= L; i++)
    for (j = 1; j <= L; j++) {
        Person[i][j].Health = Person[i][j].Swap;    // atualização sí
            ncrona
        Person[i][j].Exponent = 0;
        Person[i][j].Checked = 0;
        Person[i][j].AgeDays++;
        Person[i][j].Days++;

        if (Person[i][j].Health == Dead || Person[i][j].Health ==
            DeadCovid) {
            Person[i][j].Health = S;                // novo suscetível
            aleat();
            Person[i][j].AgeYears = rn * 100;        // idade
                reamostrada
            // ... sorteia a idade de morte natural (amostragem por
                rejeição)
        }
    }
}

```

```
}
```

Listing 5.8 – Atualização síncrona e substituição dos mortos (`Updatefunc`).

Os resultados são registrados em dois níveis. Para cada simulação, a prevalência e a incidência de cada estado de saúde são gravadas, a cada dia, em arquivos próprios. Ao final de todas as simulações, calcula-se a média das curvas sobre as `MAXSIM` simulações, obtendo-se, pelo método de Monte Carlo, as curvas médias de prevalência e de incidência que representam a evolução esperada da epidemia.

6 Implementação em GPU

A implementação paralela do modelo em CUDA constitui a principal contribuição deste trabalho. O mesmo modelo descrito no Capítulo 4 e a mesma lógica da implementação serial são reorganizados de modo que a rede seja processada em paralelo, com uma thread responsável por cada indivíduo. O desafio central é preservar o comportamento do modelo — e, portanto, a equivalência com o serial — ao mesmo tempo em que se explora o paralelismo da GPU.

6.1 Mapeamento da rede em threads

Na implementação serial, um único processador percorre a rede com dois laços encaixados e, para cada indivíduo, despacha a função correspondente ao seu estado. Na GPU, esse percurso sequencial dá lugar ao paralelismo: a cada indivíduo é associada uma thread, de modo que toda a rede é processada simultaneamente. Como a rede possui $(L + 2)^2$ posições, são lançadas $(L + 2)^2$ threads, organizadas em blocos de tamanho fixo; cada thread identifica a sua posição a partir dos índices nativos de CUDA e converte esse índice linear na posição (i, j) da matriz, ignorando as células da borda. A Listagem 6.1 apresenta as funções de conversão entre o índice da thread e a posição na rede.

```
__host__ __device__ int to1D(int i, int j, int L) {  
    return i * (L + 2) + j;  
}  
  
__host__ __device__ void to2D(int idx, int L, int &i, int &j) {  
    i = idx / (L + 2);  
    j = idx % (L + 2);  
}
```

Listing 6.1 – Conversão entre o índice da thread e a posição na rede.

Outra diferença em relação ao serial está no tratamento dos estados de saúde. Em vez de uma única varredura com um despacho condicional, cada estado possui o seu próprio kernel, lançado sobre toda a rede: cada thread verifica se o seu indivíduo se encontra no estado tratado por aquele kernel e, em caso afirmativo, aplica a regra correspondente. Os kernels de estado são lançados em sequência a cada dia — contorno, S, E, IP, IS, H, UTI e atualização —; uma vez que o lançamento de cada kernel é um ponto de sincronização global, a ordem das funções de estado do serial é preservada, o que é essencial para a equivalência entre as implementações.

6.2 Estrutura de dados na GPU

Cada indivíduo é representado na GPU por uma estrutura análoga à do serial, alinhada em 16 bytes para favorecer o acesso à memória, e a rede é armazenada como um vetor dessas estruturas na memória global do *device*. Os parâmetros do modelo — a infecciosidade, os parâmetros de cada cidade, as durações dos estados e os códigos dos estados de saúde — são armazenados em memória constante, apropriada para dados apenas de leitura que são acessados por todas as threads. A Listagem 6.2 apresenta a estrutura e alguns desses parâmetros.

```
struct __align__(16) GPUPerson {
    int Health, Swap, Gender;
    int AgeYears, AgeDays, AgeDeathYears, AgeDeathDays;
    int StateTime, TimeOnState, Days;
    int Isolation, Exponent, Checked;
};

// parâmetros e códigos de estado em memória constante
__constant__ double d_Beta;
__constant__ int d_Density;
__constant__ double d_MaxRandomContacts, d_MinRandomContacts;
__constant__ int d_S, d_E, d_IP, d_IA, d_ISLight, d_ISModerate,
    d_ISSevere;

// vetor compacto de Health (1 byte por célula)
__device__ unsigned char* d_HealthC;
```

Listing 6.2 – Estrutura do indivíduo e parâmetros em memória constante.

6.2.1 Otimização da estrutura de Health

Os mecanismos de contágio percorrem, para cada indivíduo, diversos contatos aleatórios espalhados pela rede, lendo de cada um apenas o seu estado de saúde. Como cada estrutura completa ocupa aproximadamente 52 bytes, esses acessos aleatórios a posições distantes da memória são pouco eficientes e, para redes grandes, ultrapassam a capacidade da cache. Para contornar essa limitação, mantém-se um vetor compacto que replica apenas o campo de estado de saúde, com um byte por célula, conforme ilustra a Figura 14. Os mecanismos de contágio leem desse vetor compacto, de modo que o conjunto de dados percorrido pelos acessos aleatórios é pequeno o suficiente para permanecer na cache L2 (Capítulo 3), o que reduz o tempo gasto com acesso à memória, fator determinante para o desempenho. O vetor compacto é mantido sincronizado com o estado de saúde ao final de cada atualização.

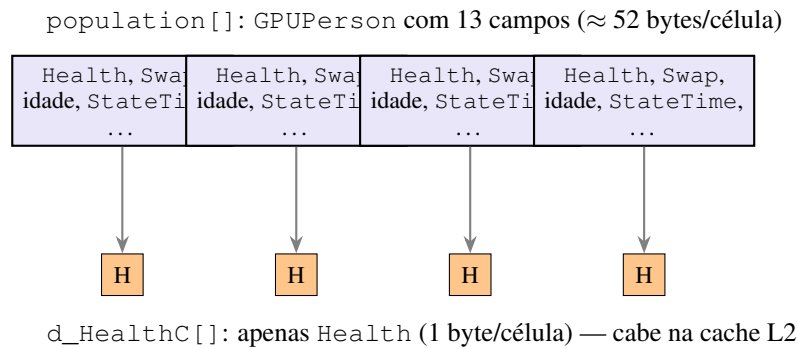


Figura 14 – Otimização da estrutura de Health. O vetor completo `population[]` armazena todos os campos de cada indivíduo; o vetor compacto `d_HealthC[]` replica apenas o campo `Health`, de modo que os acessos aleatórios dos mecanismos de contágio percorrem um vetor pequeno o suficiente para permanecer na cache L2.

6.3 Geração de números aleatórios thread-safe

A geração de números aleatórios do serial utiliza um único estado global, avançado a cada chamada, o que é incompatível com a execução paralela, uma vez que diversas threads o acessariam simultaneamente, introduzindo condições de corrida. Na implementação em GPU, mantém-se um vetor de estados, um por thread, e cada thread avança apenas o seu próprio estado, utilizando o mesmo gerador congruencial linear multiplicativo do serial. Cada estado é inicializado com uma semente própria, derivada do índice da thread, de modo que as threads produzem sequências independentes, sem condições de corrida, e os resultados permanecem reprodutíveis. A Listagem 6.3 apresenta o gerador e a sua inicialização.

```
__device__ double generateRandom(unsigned int* state) {
    unsigned long long t = (unsigned long long) (*state) * 888121ULL
        ;
    *state = (unsigned int) (t & 0xFFFFFFFF);    // mantém 32 bits,
        como no serial
    return (double) (*state) / 4294967295.0;
}

__global__ void initRNG(unsigned int* states, unsigned int seed,
    int size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < size)
        states[idx] = seed * (idx + 1);          // semente única por
        thread
}
```

Listing 6.3 – Gerador de números aleatórios por thread.

6.4 Inicialização e laço de simulação

A orquestração da simulação é feita pelo *host*, que aloca a memória do *device*, executa o laço sobre as simulações e, em cada simulação, inicializa os estados do gerador, a população e os casos iniciais. A inicialização da população é realizada em paralelo por um kernel, no qual cada thread coloca o seu indivíduo no estado suscetível e sorteia a sua idade e a sua idade de morte natural, e os 5 casos iniciais são posicionados em seguida. A cada dia, o *host* lança a sequência de kernels que constitui um passo da simulação, conforme apresenta a Listagem 6.4. Ao final de todas as simulações, os contadores são copiados do *device* para o *host*, que calcula a média das curvas sobre as simulações, do mesmo modo que na implementação serial.

```
void runSimulationDay(GPUPerson* d_pop, unsigned int* d_rng,
                    int L, int day, int blockSize, int numBlocks)
{
    updateBoundaries_kernel <<<numBlocks, blockSize>>> (d_pop, L);
    S_kernel <<<numBlocks, blockSize>>> (d_pop, d_rng, L);
    E_kernel <<<numBlocks, blockSize>>> (d_pop, d_rng, L);
    IP_kernel <<<numBlocks, blockSize>>> (d_pop, d_rng, L);
    IS_kernel <<<numBlocks, blockSize>>> (d_pop, d_rng, L);
    H_kernel <<<numBlocks, blockSize>>> (d_pop, d_rng, L);
    ICU_kernel <<<numBlocks, blockSize>>> (d_pop, d_rng, L);
    update_kernel <<<numBlocks, blockSize>>> (d_pop, d_rng, L, day,
        d_ProbNaturalDeath);
}
```

Listing 6.4 – Sequência de kernels lançada a cada dia.

6.5 Mecanismos de contágio em paralelo

O primeiro mecanismo é realizado pelo kernel do estado suscetível: cada thread cujo indivíduo é suscetível verifica a sua vizinhança e os seus contatos aleatórios, conta os infectados encontrados e, a partir da probabilidade de contágio, decide se o indivíduo passa ao estado exposto. A Listagem 6.5 apresenta esse kernel.

```
__global__ void S_kernel(GPUPerson* population, unsigned int*
rngStates, int L) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= (L + 2) * (L + 2)) return;

    int i, j;
    to2D(idx, L, i, j);
```

```

if (i < 1 || i > L || j < 1 || j > L) return;    // ignora a
        borda

int p = to1D(i, j, L);
if (population[p].Health != d_S) return;

if (population[p].Days >= population[p].AgeDeathDays) {    //
        morte natural
        population[p].Swap = d_Dead;
        return;
    }
    // busca infectados na vizinhança e nos contatos aleatórios
    ContactResult c = checkAllContacts(i, j, L, population, &
        rngStates[idx]);
    if (c.anyContact &&
        generateRandom(&rngStates[idx]) <=
            calculateInfectionProbability(c.infectiousContacts)) {
        population[p].Checked = 1;
        population[p].Swap = d_E;
        population[p].TimeOnState = 0;
        population[p].StateTime =
            generateRandom(&rngStates[idx]) * (d_MaxLatency -
                d_MinLatency) + d_MinLatency;
    } else {
        population[p].Swap = d_S;
    }
}

```

Listing 6.5 – Primeiro mecanismo: kernel do estado suscetível.

O segundo mecanismo é realizado por uma função executada pelos kernels dos estados pré-sintomático, assintomático e sintomático leve, enquanto o indivíduo permanece infeccioso: a função sorteia contatos aleatórios e tenta infectar os suscetíveis encontrados. Aqui surge o principal cuidado da paralelização: como diversas threads de indivíduos infectados podem mirar o mesmo suscetível simultaneamente, o incremento simples do campo `Exponent` usado no serial é substituído por uma operação atômica, que garante que exatamente uma dessas threads efetive a infecção. A Listagem 6.6 apresenta essa função.

```

__device__ void spreadInfection_kernel(int i, int j, GPUPerson*
    population,

                                unsigned int* rngState, int
                                L) {

```

```

double rn = generateRandom(rngState);
int randomContacts = rn * (d_MaxRandomContacts -
    d_MinRandomContacts) + d_MinRandomContacts;

for (int c = 0; c < randomContacts; c++) {
    int Randomi = (int) (generateRandom(rngState) * L) + 1;    //
        sorteia um alvo
    int Randomj = (int) (generateRandom(rngState) * L) + 1;
    int r = tolD(Randomi, Randomj, L);

    if (d_HealthC[r] == d_S) {
        // deduplicação concorrente: incremento atômico de
        Exponent
        int oldval = atomicAdd(&population[r].Exponent, 1);
        if (population[r].Checked == 0 && oldval == 0) {
            double P = 1.0 - pow(1.0 - d_Beta, (double) (oldval
                + 1));
            if (generateRandom(rngState) <= P) {
                population[r].Checked = 1;
                population[r].Swap = d_E;
                population[r].TimeOnState = 0;
                population[r].StateTime =
                    generateRandom(rngState) * (d_MaxLatency -
                        d_MinLatency) + d_MinLatency;
            }
        }
    }
}

```

Listing 6.6 – Segundo mecanismo: deduplicação concorrente por operação atômica.

6.6 Atualização e contadores

A atualização é realizada por um kernel no qual cada thread copia o campo `Swap` para o campo `Health` do seu indivíduo, incrementa os contadores de idade e, quando o indivíduo sofreu morte natural, substitui-o por um novo suscetível com idade reamostrada, de modo análogo ao serial. Ao final, cada thread mantém o vetor compacto de `Health` sincronizado. A contagem da prevalência e da incidência de cada estado, que no serial é feita por um laço sequencial, na GPU é realizada por operações atômicas sobre contadores no *device*, uma vez que todas as threads escrevem nesses contadores simultaneamente; os contadores são então copiados

para o *host*. A Listagem 6.7 apresenta o trecho central desse kernel.

```
__global__ void update_kernel(GPUPerson* population, unsigned int*
    rngStates,
                                int L, int day, double*
                                probNaturalDeath) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= (L + 2) * (L + 2)) return;
    int i, j; to2D(idx, L, i, j);
    if (i < 1 || i > L || j < 1 || j > L) return;
    int p = to1D(i, j, L);

    population[p].Health = population[p].Swap;    // atualização síncrona
    population[p].AgeDays++;
    population[p].Days++;

    // morte natural: substitui por um novo suscetível
    if (population[p].Health == d_Dead || population[p].Health ==
        d_DeadCovid)
        replaceDeadPerson(&population[p], &rngStates[idx],
            probNaturalDeath);

    // contagem da prevalência por operações atômicas
    int s = population[p].Health;
    if (s == d_S) atomicAdd(&d_S_Total, 1);
    else if (s == d_E) atomicAdd(&d_E_Total, 1);
    // ... demais estados

    population[p].Exponent = 0;
    population[p].Checked = 0;
    d_HealthC[p] = (unsigned char) s;    // mantém o vetor compacto
                                         sincronizado
}
```

Listing 6.7 – Atualização síncrona, substituição e contagem por operações atômicas.

7 Resultados e Discussão

Este capítulo apresenta os resultados obtidos seguindo a estratégia estabelecida na Seção 4: primeiro, caracteriza-se a evolução da epidemia produzida pelo modelo nas quatro cidades; em seguida, valida-se a implementação em GPU comparando-a com a serial sob os mesmos parâmetros; e, por fim, avalia-se o ganho de desempenho da paralelização. Os resultados foram obtidos com $\text{MAXSIM} = 50$ simulações por cidade, ao longo de 400 dias. O ambiente computacional utilizado nos experimentos é descrito na Seção 7.1.

7.1 Ambiente computacional

Os experimentos principais foram realizados em uma máquina na qual a implementação serial foi executada em um único núcleo da CPU, servindo de referência, e a implementação paralela foi executada na GPU. A Tabela 4 resume as características de hardware das plataformas utilizadas, incluindo a segunda GPU empregada apenas na análise comparativa entre placas.

	CPU (serial)	GPU principal	GPU (comparação)
Modelo	AMD Ryzen 7 5700X	RTX 4070 SUPER	GTX 1050 Ti
Arquitetura	Zen 3	Ada Lovelace (sm_89)	Pascal (sm_61)
Núcleos	8 núcleos / 16 threads	7168 CUDA cores	768 CUDA cores
Frequência base	3,4 GHz	—	—
Memória	16 GB DDR4	12 GB GDDR6X	4 GB GDDR5
Banda de memória	—	504 GB/s	112 GB/s
Cache L2	—	48 MB	1 MB

Tabela 4 – Características de hardware das plataformas utilizadas.

Embora a CPU disponha de 8 núcleos, a implementação serial utiliza um único núcleo, de modo a constituir a referência sequencial contra a qual o ganho da paralelização é medido. Ambas as implementações foram compiladas com o compilador nvcc (CUDA 12.6) e otimização ativada, sob o sistema operacional Windows 11. A GPU GTX 1050 Ti, de arquitetura anterior e com largura de banda de memória cerca de quatro vezes menor, é utilizada exclusivamente na análise comparativa entre placas (Figura 23), que confirma a natureza limitada por memória da simulação.

7.2 Evolução da epidemia nas cidades modeladas

A Figura 15 apresenta a prevalência de cada estado de saúde ao longo do tempo para as quatro cidades. Em todas elas, a população, inicialmente suscetível, é progressivamente infectada

à medida que a epidemia avança, atingindo um pico de indivíduos infecciosos e estabilizando-se em um patamar final de recuperados e de suscetíveis remanescentes. A proporção de indivíduos que contraíram a doença ao final da simulação, ou seja, a taxa de ataque, varia de aproximadamente 59% na Rocinha a cerca de 77% em Brasília, enquanto a mortalidade por Covid-19 situa-se entre 10% e 13% da população.

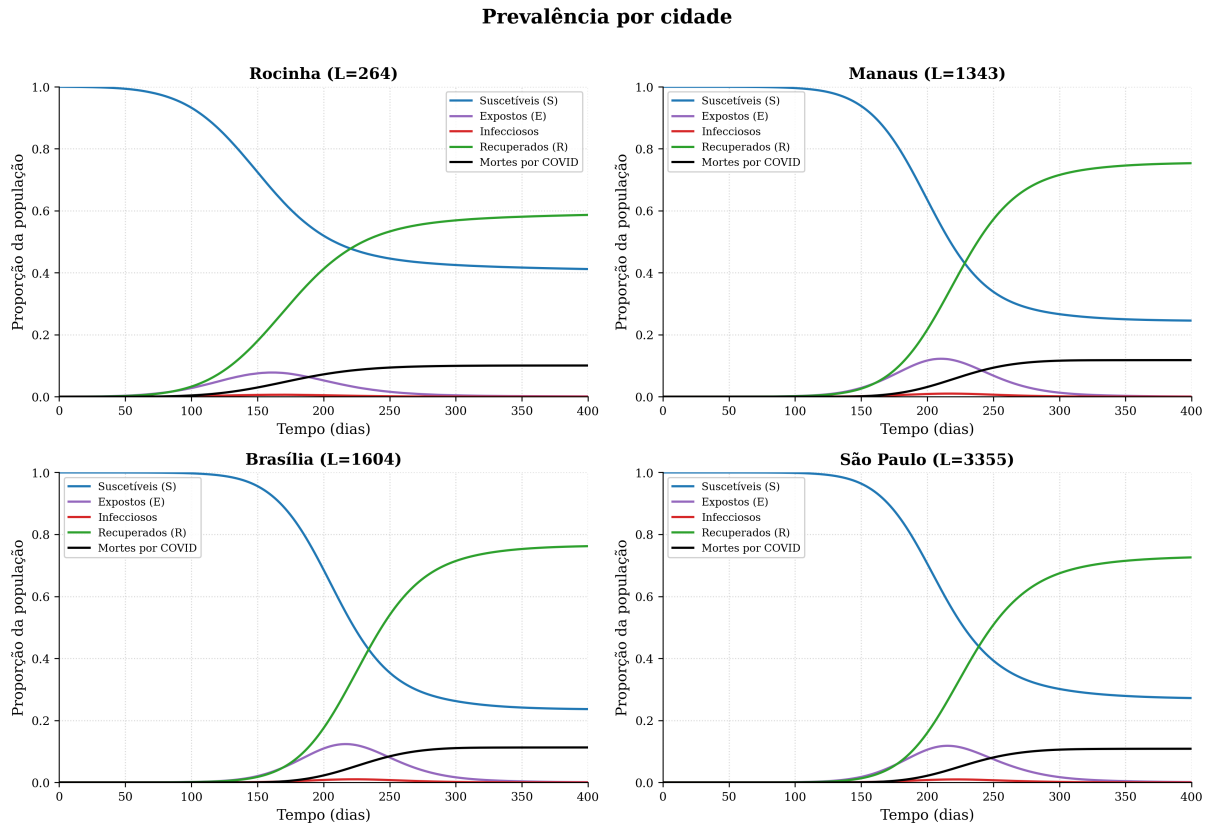


Figura 15 – Prevalência de cada estado de saúde ao longo do tempo, para as quatro cidades modeladas.

A Figura 16 detalha a carga clínica imposta por cada epidemia, mostrando a prevalência de infecciosos e a ocupação de leitos de hospital e de UTI. Observa-se que a proporção de indivíduos hospitalizados e em UTI é muito menor que a de infecciosos, porém persiste por um período prolongado, o que reflete a maior duração desses estados.

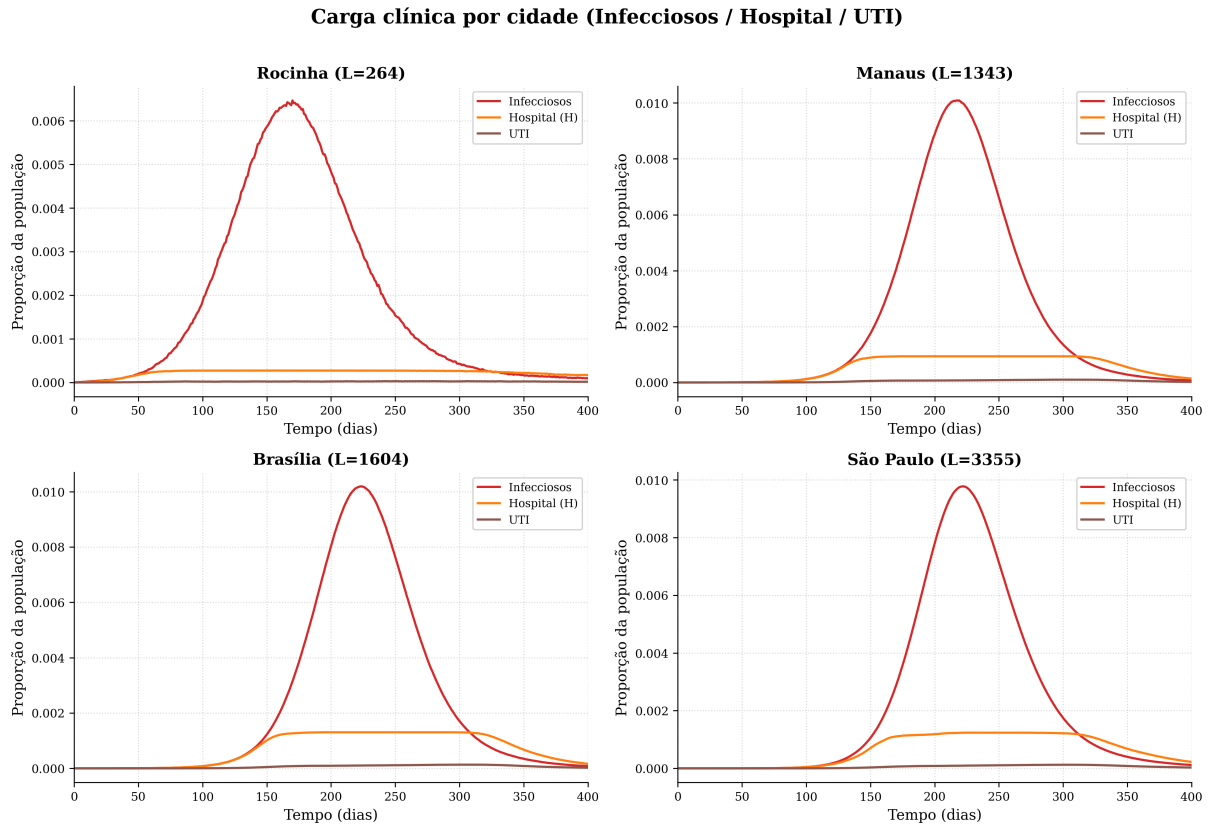


Figura 16 – Carga clínica por cidade: prevalência de infecciosos e ocupação de leitos de hospital e de UTI.

7.3 Validação da implementação paralela

A validação da implementação em GPU consiste em verificar se ela reproduz os resultados da implementação serial, tomada como referência, quando ambas são executadas sob os mesmos parâmetros. A Figura 17 sobrepõe as curvas de prevalência obtidas pelas duas implementações para cada cidade, com a curva serial em traço sólido e a curva da GPU em traço tracejado. As curvas são praticamente indistinguíveis, o que indica que a paralelização preservou o comportamento do modelo.

Prevalência – Serial × GPU por cidade

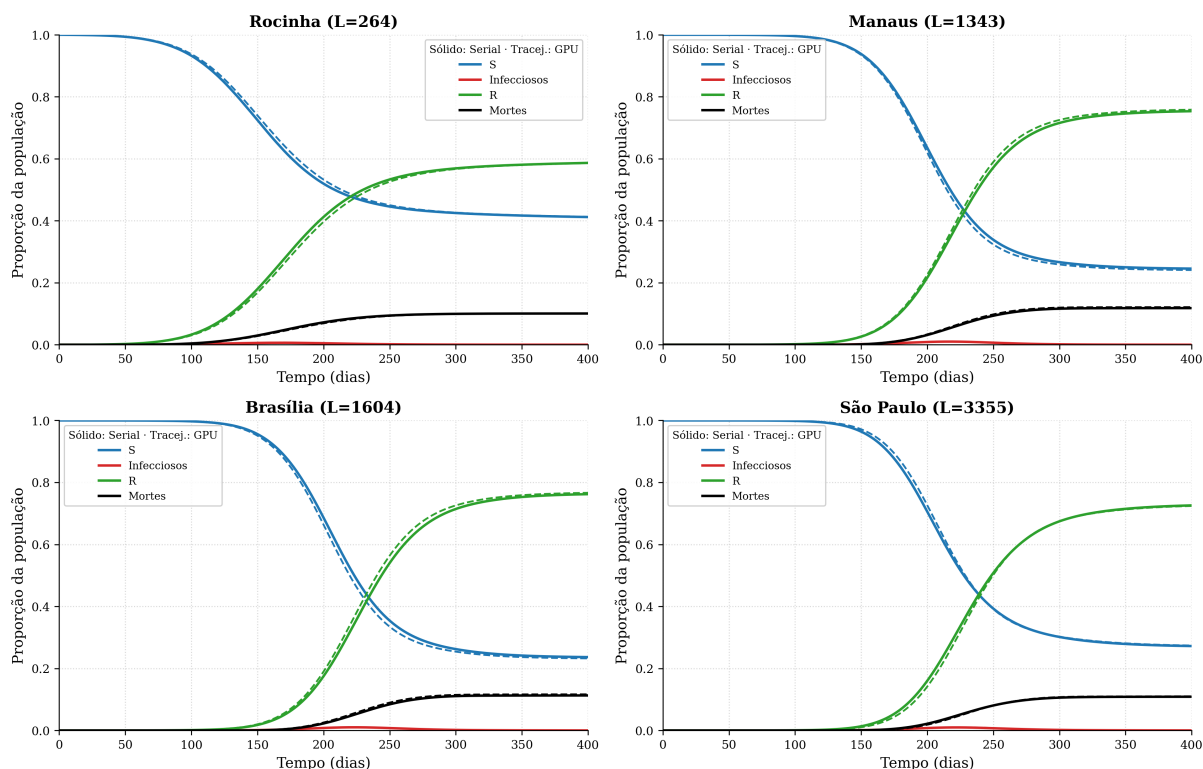


Figura 17 – Curvas de prevalência das implementações serial (traço sólido) e em GPU (traço tracejado), sob o mesmo β , para as quatro cidades.

A concordância entre as implementações é quantificada pelo erro quadrático médio (RMSE) entre as curvas de proporção de suscetíveis ao longo do tempo. A Tabela 5 apresenta, para cada cidade, a infecciosidade β calibrada para $R_0 = 3,5$, a taxa de ataque obtida por cada implementação e o RMSE correspondente.

Cidade	β	Ataque (serial)	Ataque (GPU)	RMSE de $S(t)$
Rocinha	0,0049	58,8%	58,7%	0,76 p.p.
Manaus	0,0995	75,4%	75,9%	0,86 p.p.
Brasília	0,0995	76,3%	76,8%	1,13 p.p.
São Paulo	0,0243	72,8%	72,6%	0,74 p.p.

Tabela 5 – Validação da implementação em GPU contra a serial, sob o mesmo β .

Em todas as cidades, o RMSE é inferior a 1,2 pontos percentuais, conforme ilustra a Figura 18. Uma diferença dessa ordem é compatível com a flutuação estatística inerente ao método de Monte Carlo com um número finito de simulações, uma vez que cada implementação utiliza a sua própria sequência de números pseudoaleatórios. Conclui-se, portanto, que as duas implementações são equivalentes, e logo a paralelização é válida.

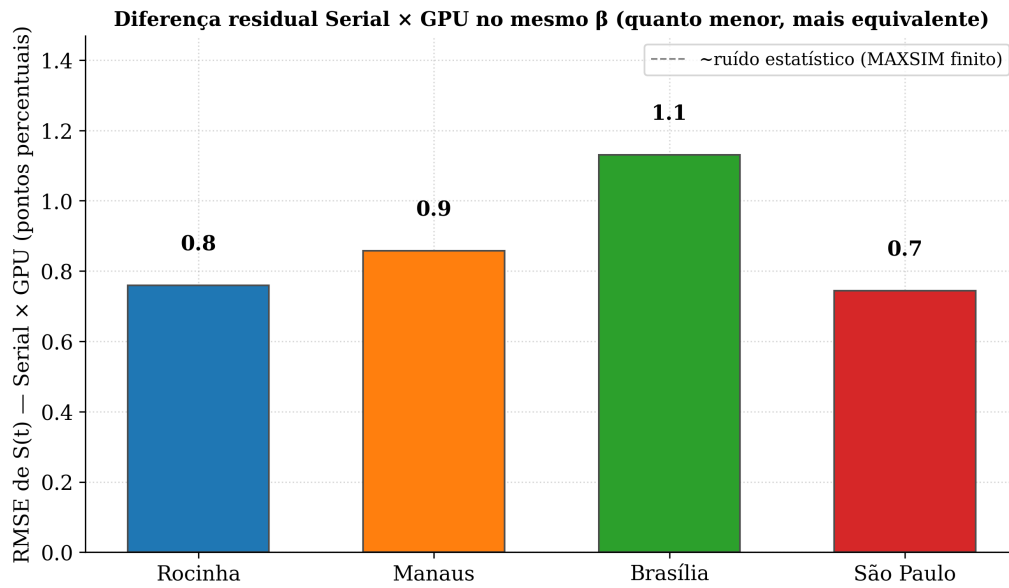


Figura 18 – Diferença residual (RMSE de $S(t)$) entre as implementações serial e em GPU, sob o mesmo β , para as quatro cidades.

7.4 Desempenho

7.4.1 Aceleração e eficiência

Confirmada a equivalência, avalia-se o ganho de desempenho da implementação em GPU. A Tabela 6 apresenta, para cada cidade, o tamanho da rede, o tempo total de execução de cada implementação e a aceleração obtida, e a Figura 19 apresenta a aceleração por cidade.

Cidade	L	Células	Serial	GPU	Aceleração
Rocinha	264	70 mil	235 s	129 s	1,8×
Manaus	1343	1,80 mi	1.891 s	136 s	13,9×
Brasília	1604	2,57 mi	2.124 s	131 s	16,2×
São Paulo	3355	11,25 mi	35.755 s	634 s	56,4×

Tabela 6 – Tempo de execução e aceleração da implementação em GPU sobre a serial (50 simulações, 400 dias).

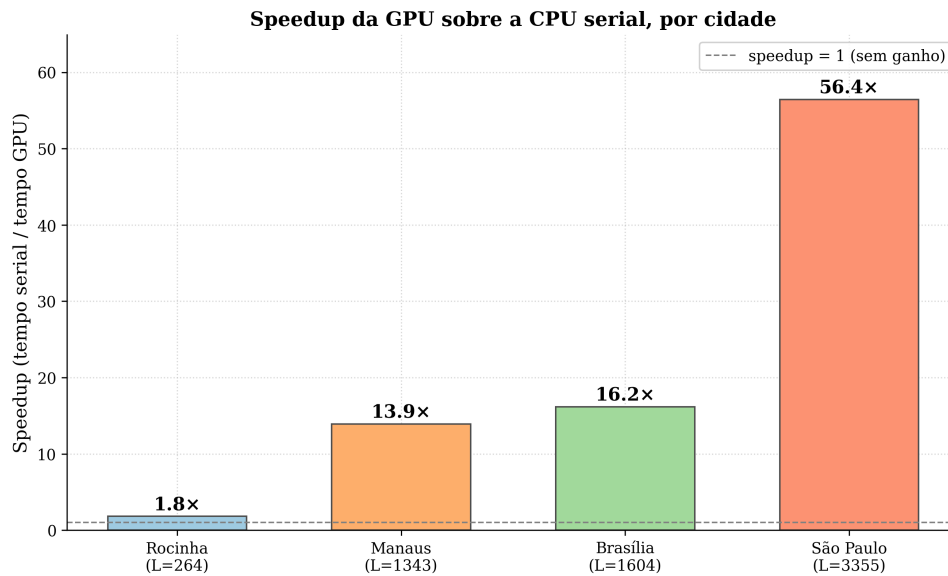


Figura 19 – Aceleração da implementação em GPU sobre a serial, por cidade.

O ganho de desempenho é expressivo nas cidades grandes: em São Paulo, a maior rede, a simulação que levava cerca de 9,9 horas em CPU passa a ser executada em pouco mais de 10 minutos na GPU, uma aceleração de 56,4 $\times$. Na Rocinha, a menor rede, a aceleração é de apenas 1,8 $\times$.

7.4.2 Escala com o tamanho da rede

A aceleração obtida cresce com o tamanho da rede, como mostra a Figura 20. Esse comportamento decorre da natureza da GPU: o seu desempenho só é plenamente aproveitado quando há trabalho suficiente para manter os milhares de threads ocupados. Em redes pequenas, como a da Rocinha, há poucas células e, portanto, poucas threads, de modo que grande parte do potencial da GPU permanece ociosa e o ganho é modesto; em redes grandes, como a de São Paulo, a quantidade de threads é suficiente para saturar o paralelismo da GPU, e logo a aceleração é muito maior.

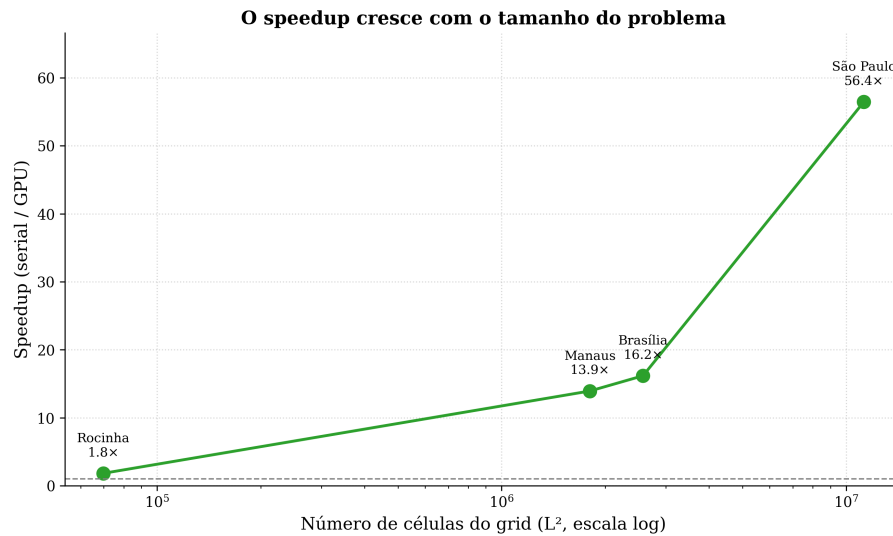
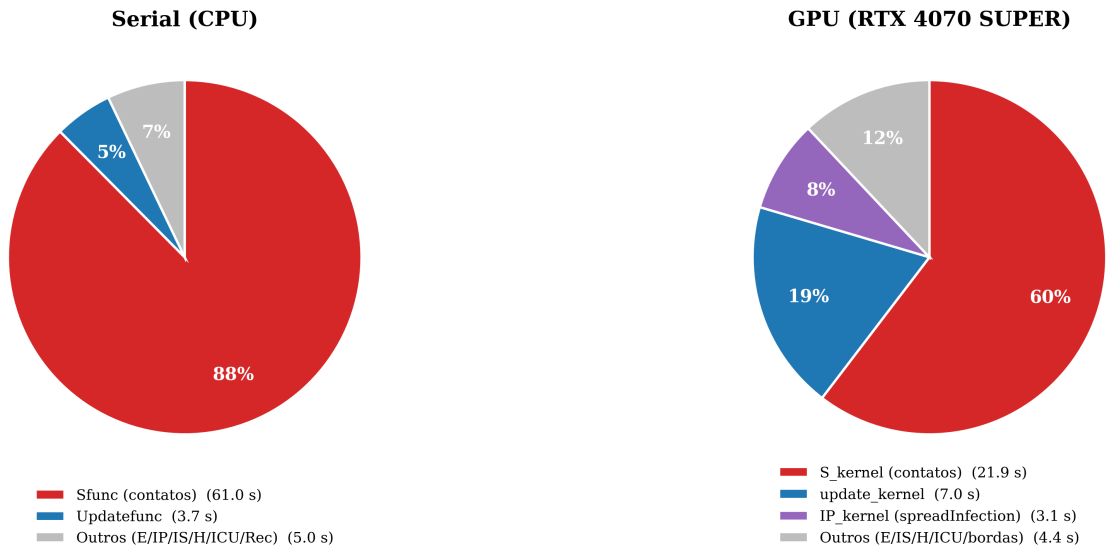


Figura 20 – Aceleração em função do número de células da rede.

7.4.3 Análise do desempenho

Para compreender o que determina o desempenho, analisou-se a distribuição do tempo de execução entre as diferentes etapas da simulação. A Figura 21 mostra que, tanto na implementação serial quanto na paralela, a verificação dos contatos do indivíduo suscetível domina o tempo de execução, respondendo por 88% do tempo no serial e por 60% na GPU. Essa etapa é justamente a que realiza acessos aleatórios à memória, ao percorrer contatos espalhados pela rede.

Perfil de execução: onde o tempo é gasto



Em ambas, a verificação de contatos do suscetível (S) é o gargalo — acesso aleatório à memória.

Figura 21 – Distribuição do tempo de execução entre as etapas da simulação, nas implementações serial e em GPU.

Esse predomínio dos acessos aleatórios à memória indica que a simulação é limitada por memória (*memory-bound*), o que justifica a otimização da estrutura de Health descrita no Capítulo 6. A Figura 22 mostra o impacto dessa otimização: em São Paulo, cidade com grande rede e muitos contatos, o vetor compacto reduz o tempo por simulação de 160 segundos para cerca de 12 segundos, uma aceleração de aproximadamente $14\times$; em Manaus, cidade com poucos contatos, o efeito é desprezível, uma vez que os acessos aleatórios já são pouco numerosos. Esse contraste confirma que o gargalo está no acesso à memória durante a verificação dos contatos.

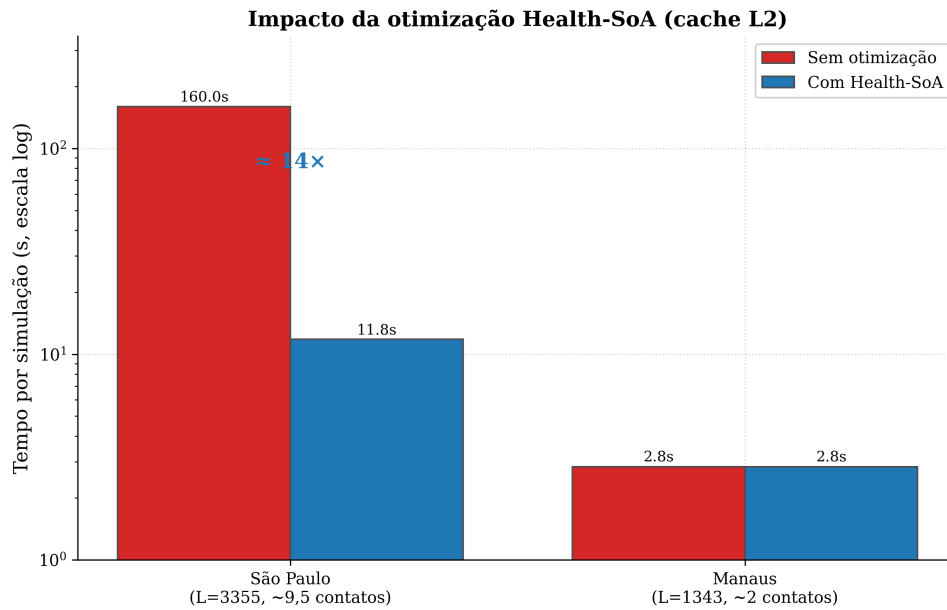


Figura 22 – Impacto da otimização da estrutura de Health sobre o tempo por simulação.

Uma confirmação independente de que a simulação é limitada por memória é obtida comparando-se o desempenho em duas GPUs distintas, a RTX 4070 SUPER e uma GTX 1050 Ti, conforme a Figura 23. Executando o mesmo código nas duas placas, a razão entre os tempos nas cidades grandes aproxima-se da razão entre as larguras de banda de memória das GPUs (504 GB/s e 112 GB/s, uma razão de 4,5 $\times$), e não da razão entre as suas capacidades de cálculo em ponto flutuante (17 $\times$); ou seja, o desempenho acompanha a banda de memória, e não o poder de cálculo, o que caracteriza um problema limitado por memória.

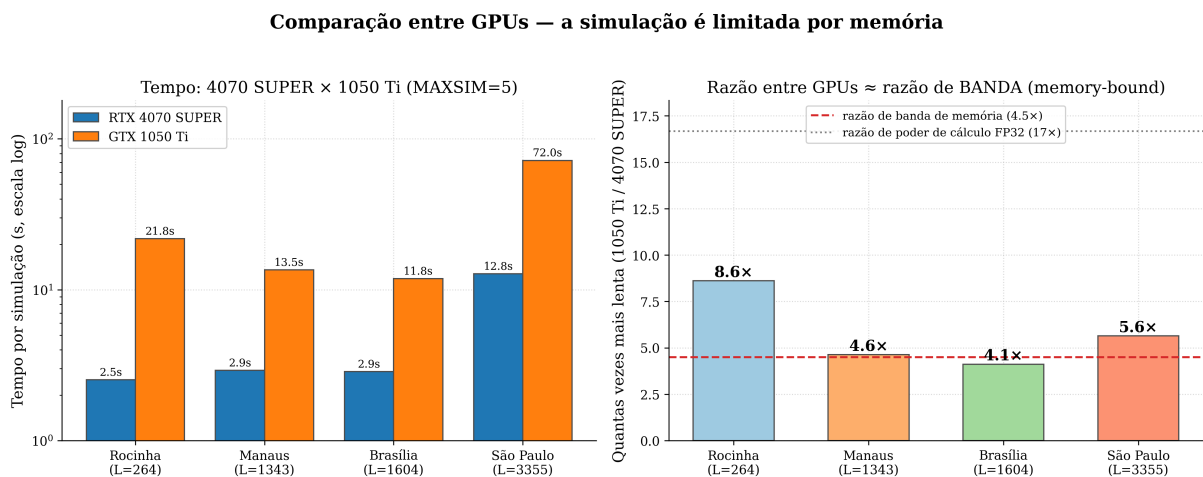


Figura 23 – Comparação de desempenho entre duas GPUs. A razão entre os tempos acompanha a razão entre as larguras de banda de memória, confirmando que a simulação é limitada por memória.

7.5 Variabilidade dos resultados

Como o modelo é estocástico, cada simulação produz uma realização distinta da epidemia, e os resultados apresentados correspondem à média sobre as 50 simulações. A Figura 24 mostra, para cada cidade, a média da proporção de suscetíveis acompanhada da banda de um desvio-padrão em torno dela. As bandas são estreitas, o que indica que, no regime calibrado para $R_0 = 3,5$, as simulações individuais variam pouco em torno do comportamento médio e os resultados são estatisticamente robustos.

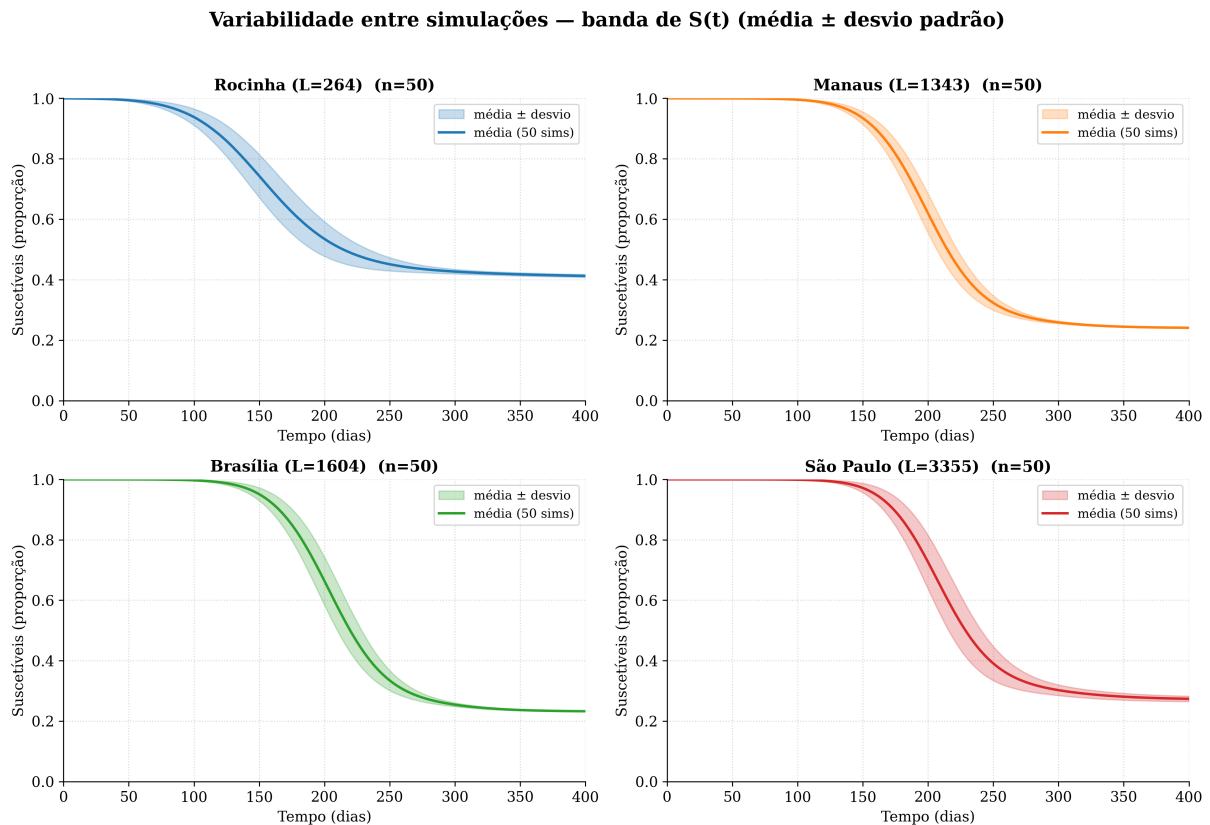


Figura 24 – Média da proporção de suscetíveis e banda de um desvio-padrão sobre as 50 simulações, por cidade.

Essa robustez, entretanto, depende do regime considerado. A Figura 25 compara a distribuição das taxas de ataque das 50 simulações no regime calibrado ($R_0 = 3,5$) e em um regime próximo ao limiar epidêmico. No regime calibrado, as taxas de ataque concentram-se em torno da média, com desvio-padrão de cerca de 0,5 ponto percentual; já próximo ao limiar, a incerteza é muito maior, com desvio de cerca de 3,6 pontos percentuais e a ocorrência de extinções estocásticas da epidemia em algumas simulações. Conclui-se que os resultados reportados neste trabalho, obtidos no regime calibrado, são estatisticamente robustos.

Rocinha — incerteza alta perto do limiar × robustez no regime calibrado

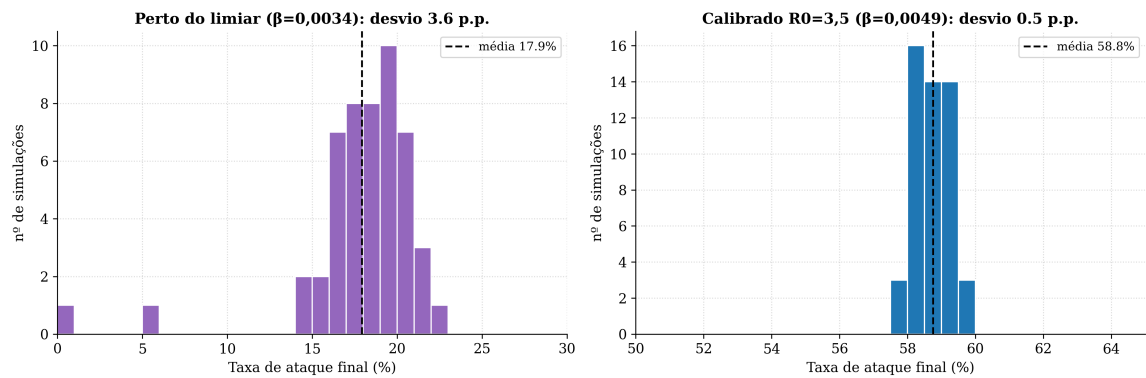


Figura 25 – Distribuição das taxas de ataque das 50 simulações no regime calibrado ($R_0 = 3,5$) e próximo ao limiar epidêmico.

Referências

- [1] Merrill R. Introduction to Epidemiology. Jones & Bartlett Learning: London, United Kingdom, 2010.
- [2] Keeling, M.J. and Rohani, P. (2008) Modeling infectious diseases in humans and animals. Princeton: Princeton University Press.
- [3] Sanders, J. and Kandrot, E. (2011) Cuda by example: An introduction to general-purpose GPU programming. Upper Saddle River: Addison-Wesley.
- [4] Cheng, J., Grossman, M. and McKercher, T. (2014) Professional CUDA® C programming. Indianapolis, IN: Wrox, a Wiley brand.
- [5] http://cnes2.datasus.gov.br/Mod_Ind_Tipo_Leito.asp?VEstado=00
- [6] <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>
- [7] <https://docs.nvidia.com/cuda/curand/index.html>